



C.P.R. Liceo “La Paz”

Proyecto Fin de Ciclo

Desarrollo de Aplicaciones Multiplataforma

Autor: Álex Antelo Fernández

Tutor: Jesús Ángel Pérez-Roca Fernández

Resumen

FitTrack es una aplicación desarrollada para Android destinada a las personas del mundo “*fitness*”. La aplicación permite el seguimiento de las rutinas deportivas para facilitar el día a día de un entrenamiento y tener a mano la información necesaria para el desenvolvimiento del mismo. No limitándose solo a las rutinas y ejercicios la aplicación cuenta con añadidos extra, todos relacionados con el mundo del deporte y sus necesidades.

El funcionamiento es simple, contando con una estructura y uso muy intuitivos se podrá, fácilmente, crear y gestionar las rutinas. Estas rutinas constarán de una lista de ejercicios asociados a ellas, pudiendo modificar esta lista en cualquier momento según lo que se necesite.

Las rutinas son completamente personalizadas, los ejercicios pueden ser tanto creados por el usuario como seleccionados en una serie de ejercicios comunes añadidos por la realización habitual de estos por la comunidad y por personajes influyentes del “*fitness*”.

El usuario tendrá la opción de crear sus ejercicios personalizados en los que podrá añadir desde una descripción hasta los músculos que el ejercicio comprende. Según como esta diseñada la aplicación, los ejercicios y rutinas creados por los usuarios son accesibles por el resto de la comunidad buscando crear un entorno saludable y de cooperación.

Como añadidos extra, la aplicación cuenta con una serie de funcionalidades útiles para el mundo “*fitness*” como pueden ser el cálculo de las calorías basales y de mantenimiento o un seguimiento del agua consumida en el día.

A nivel técnico la aplicación cuenta con un API creada en Spring para la persistencia de los datos y la autenticación de los usuarios. Usando Spring Security y JWT el usuario podrá acceder a su cuenta y datos desde cualquier dispositivo sin problema alguno ya que las contraseñas cuentan con un doble cifrado.

Siguiendo el sistema de roles, los “*endpoints*” estará protegidos para que los administradores de la aplicación puedan realizar cambios a gran escala mientras que los usuarios normales solamente puedan acceder y modificar sus datos. Igualmente las rutinas y los ejercicios serán accesibles para la lectura y escritura por parte de todos los usuarios pero no para el borrado, el cual solo lo podrán realizar administradores.



Abstract

FitTrack is an Android application intended for people in the fitness world. The application permits the track of sports routines to ease the day-to-day of your training and to have at hand the necessary information for its development. Not only limited to the routines and exercises, but the application also has extra functionalities, all related to the sports world and its necessities.

The operation is straightforward, incorporating an intuitive structure and usage, it will be easy to effortlessly create and manage routines. These routines will comprise a list of associated exercises, allowing for the modification of this list at any time as needed.

The routines are fully customizable, with exercises that can be either created by the user or selected from a series of common exercises added by the community and influential fitness personalities.

"The user will have the option to create custom exercises, where they can add a description as well as specify the muscles involved in the exercise. As the application is designed, the exercises and routines created by users will be accessible to the rest of the community, aiming to create a healthy and cooperative environment.

As additional features, the application includes a set of useful functionalities for the fitness world, such as calculating basal and maintenance calories or tracking daily water intake.

On a technical level, the application features an API built with Spring for data persistence and user authentication. Utilizing Spring Security and JWT, users can access their accounts and data seamlessly from any device, as passwords are double encrypted.

Following the role-based system, the endpoints will be protected to allow administrators of the application to make large-scale changes, while regular users will only have access to and be able to modify their data. Similarly, routines and exercises will be accessible for reading and writing by all users but not for deletion, which can only be performed by administrators.

Palabras Clave

- Framework – Software diseñado para facilitar el desarrollo de un tipo de aplicaciones específicas.
- Http – Protocolo de comunicaciones web.
- Spring – Framework para el diseño de aplicaciones de servicio web.
- Api Rest – Aplicación web creada para la gestión de datos a través de peticiones Http.
- SQL – Lenguaje de consultas estructurado
- Mysql – Software de bases de datos SQL.
- Android – Sistema operativo móvil.
- Singleton – Patrón de diseño que se basa en la creación de una única instancia de una clase.
- React – Framework de desarrollo Front-end.
- Java – Lenguaje y tecnología de desarrollo.
- JVM – Máquina virtual sobre la que trabaja Java.
- Kotlin – Lenguaje de desarrollo para la JVM.
- SDK – Kit para el desarrollo de software.
- Retrofit – Paquete para el uso de peticiones Http en Android.
- Token – Cadena de caracteres utilizada para autenticar peticiones.
- Endpoint – Punto de conexión a un servicio web.
- Optional – Tipo de objeto en java que contiene un objeto de otra clase o esta vacío.
- Bean – Componente inicializado en el contexto de Spring como una clase básica o un servicio entre otros.
- Github – Aplicación web para la publicación de proyectos.
- Apk – Paquete de instalación de una aplicación para android.
- Jar – Paquete de Java.



Índice

Resumen.....	3
Abstract.....	4
Palabras Clave.....	5
Introducción/motivación.....	10
Objetivos.....	11
Estado del arte.....	12
Caso de estudio.....	15
Diagramas.....	16
FitTrackApi.....	16
FitTrack Mobile.....	18
Base de datos.....	20
Caso de uso.....	21
Desarrollo del proyecto.....	22
FitTrack Mobile.....	22
Login Page.....	23
Register Page.....	24
Home.....	25
Tools.....	27
Profile.....	28
Routine.....	29
Add Exercise.....	31
Create Exercise.....	32
User Edit Page.....	33
1RM Calculator.....	34
Calories Calculator.....	35
Water Calculator.....	36
Water Register.....	37
Context.....	38
Api Manager.....	39
API.....	40
Auth.....	41



FitTrack – Aplicación de seguimiento deportivo.

Exercise.....	43
Record.....	44
Routine.....	45
User.....	46
Configuraciones.....	47
Manual Administrador.....	49
Manual Usuario.....	51
Viabilidad tecno-económica.....	57
Trabajo futuro.....	59
Conclusiones.....	60
Biblioteca de recursos web y referencias.....	61



Introducción/motivación.

El planteamiento de este proyecto viene dado por como está el sector en cuanto a aplicaciones de calidad para entrenar. Debido a mi implicación con el deporte he necesitado a lo largo de mi desempeño herramientas que fuesen buenas y cómodas para facilitarme y mejorar el seguimiento del mismo. El principal problema con el resto de aplicaciones del mercado eran los planes de pago excesivos y restrictivos, la poca personalización y una forma pobre de seguir los entrenamientos.

Enfrentándome a este problema a diario, la idea de hacer una aplicación para el seguimiento deportivo que superase a las demás rondó mi cabeza desde hace tiempo, cuando aún no tenía los conocimientos necesarios para realizarla. Si bien no estaba seguro de si ese era el proyecto que realmente haría, debido a un entorno en el que se encuentra gente de índole deportista, divagaba sobre las posibles funcionalidades y aplicaciones que podría tener mi proyecto.

Viendo todas las ideas que se me ocurrían y debido a la falta de aplicaciones competentes en el mercado, la idea fue poco a poco calando más en mi mente y según mis conocimientos crecían, veía más y más posible la realización de un proyecto que pudiese estar no solo a la altura de mis expectativas si no a las del público general. Este público, por las sensaciones que parecían tener, estaba en una situación muy parecida a la mía, por lo que mi idea siempre calaba entre ellos.

Acercándose el final de las clases sabía que no iba a tener un conocimiento más avanzado por parte del centro, ya que las clases terminaban, y que si tenía que hacer un proyecto tenía que decidir ya cual era. Irónicamente, en estas semanas de duda, una persona cercana a mí, la cual apoyaba mi idea de la aplicación de seguimiento deportivo, me mostró una aplicación nueva con la que podías hacer un seguimiento muy personalizado y exhaustivo, de la misma forma que yo había planificado. Abordé la noticia de dos formas, primeramente estuve molesto porque lo que podía empezar como un proyecto escolar podría terminar siendo un producto real y de calidad del que poder vivir tenía una dura competencia, pero por otra parte me di cuenta de lo que esa aplicación podría ayudarme para confeccionar conceptualmente el proyecto.

Tras haber usado esta aplicación mencionada anteriormente durante unas semanas y viendo lo útil que era y las facilidades que había introducido en mi vida, estaba seguro de que tenía que realizar como proyecto una aplicación similar. Por lo que tras ver un proyecto con una utilidad tan real y con un funcionamiento tan práctico lo tuve claro, mi proyecto tenía que ser “*FitTrack*”

Objetivos.

El objetivo principal de esta aplicación es competir con los actuales líderes en el mercado de las aplicaciones deportivas, ofreciendo una alternativa igual de fiable y además gratuita. No obviando la calidad, al ser una alternativa gratuita recaerá su desarrollo en la funcionalidad antes que en un diseño complejo y altamente responsivo.

Estando en contacto directo con los usuarios, se quiere formar una comunidad muy cercana en la que la gente coopere para conseguir sus metas personales, para que conecte con personas con la misma pasión que ellos y que nos ayuden a construir un entorno deportivo muy saludable. Siempre confiando en nuestros usuarios y debido a la naturaleza de las personas del mundo deportivo, las opinión y sugerencias serán una pieza clave para llevar la aplicación al siguiente nivel.

También se busca tener una experiencia completa del deporte agregando todas esas funcionalidades que puedan rodear y rodeen el desarrollo deportivo, como puede ser un seguimiento alimenticio y de hidratación entre otros.

“FitTrack” aspira a ser la mejor herramienta de bolsillo del deportista siendo indispensable para realizar un seguimiento de calidad, no solo a nivel personal si no como colectivo. Creando esta comunidad mencionada anteriormente se abrirá la posibilidad a que equipos y entrenadores de grupos puedan usar su aplicación durante el día a día para desarrollar su trabajo y entrar en contacto con su equipo de una forma más cercana y directa.

Como último punto de la aplicación, una de las cosas esenciales para el deporte es el progreso. Ver hacía el pasado y darte cuenta de tu progreso es uno de los puntos más esenciales y gratificantes del deporte, por lo que brindarles a los usuarios una serie de informes y estadísticas con los datos que ellos nos han entregado es uno de los objetivos más importantes.

Finalmente, como el objetivo más importante del proyecto es mantener la cultura del deporte y facilitar mucho la entrada a toda persona que quiera introducirse en él. Apoyar la cultura del esfuerzo y el progreso y sobretodo, la cooperación.

Estado del arte.

En el mercado existen diversidad de aplicaciones similares para suplir este problema, pero hay una que destaca sobre el resto y me voy a centrar en ella, “*Hevy*”.

“*Hevy*” te brinda una interfaz muy atractiva y fácil de usar. Con sus funciones bien delimitadas, realiza su trabajo y bien. Su función principal y única es la confección de rutinas a las que darles seguimiento, de forma individualizada, por ejercicio, y a nivel de progreso global.

“*Hevy*” también da unas estadísticas muy bien formateadas y presentadas para que puedas ver bien tu progreso a lo largo del tiempo como los músculos que atacas en cada sesión de entrenamiento. Este seguimiento también incluye un sistema de racha de semanas de entreno y una imagen de los músculos entrenados a lo largo de los 7 últimos días entre otros gráficos.

Una de las cosas más interesantes de la que carecen la mayoría de aplicaciones del mercado es en la forma de añadir series y establecer temporizadores de descanso entre estas para cada ejercicio. También, si en algún momento necesitas recordar como se realiza un ejercicio puedes entrar en la vista de este ejercicio, donde te mostrará tu histórico y una explicación de como se realiza además de un vídeo.

Siendo una aplicación tan completa y líder, es complicado encontrarle puntos negativos sin compararlo con una aplicación que esté por encima pero hay algunos puntos que es importante tener en cuenta.

Si bien el seguimiento de la rutina es muy completo, en el plan gratuito solamente puedes seguir simultáneamente cuatro rutinas, que es verdaderamente poco ya que aunque entrenes menos de cuatro días, si quisieras tener dos versiones de un mismo día de entrenamiento tendrías que sacrificar un espacio. Esto también choca con una de sus funcionalidades principales que es la sección de red social.

En cuestión de añadidos para el entorno del deporte, “*Hevy*” es bastante pobre, no teniendo nada más allá del seguimiento deportivo. Si bien llevar cuenta de tu progreso es muy importante, hay muchas otras cosas para las que tendrás que depender de otra aplicación externa, cuando podría estar incluidas.

Al estilo de otras aplicaciones como *"FatSecret"*, *"Hevy"* tiene una sección social. En esta sección los usuarios interactúan entre ellos, haciendo publicaciones y comentando, pero sobretodo, comparten rutinas o ejercicios que pueden ser añadidos a tu repertorio, siendo una función muy útil pero limitando aún más estos cuatro espacios que mencionaba.

Como punto de partida, *"FitTrack"* tendrá todas las funcionalidades que *"Hevy"* trae consigo y mucho más. La principal mejora es un límite de rutinas posibles por usuario mucho mayor, que se defina no por plan de pago si no por limitaciones técnicas.

Otro añadido importante será el diseño enfocado a grupos. Al ser uno de nuestros objetivos brindarle a los equipos deportivos una forma de crear los entrenamientos de forma colectiva, no solo será una opción interesante para apasionados del mundo, si no también para profesionales federados que necesiten una aplicación potente para llevar sus entrenamientos a otro nivel.

Unas mejoras sustanciales que tendrá además de las mencionadas son todas las funcionalidades del entorno deportivo, que irán creciendo más y más según los usuarios las requieran y perfeccionándose hasta ser referente en el sector.

A nivel técnico, la aplicación es en Android. Al ser un entorno tan específico la mayor parte de aplicaciones usan tecnologías muy parecidas para el desarrollo. En este caso la aplicación es desarrollada en Java y *"Kotlin"*. Una opción interesante que podría ser materia de estudio es *"React Native"*, por su potencia y compatibilidad.

Teniendo todos estos puntos en cuenta, se ha creado el siguiente análisis DAFO (Debilidades, amenazas, fortalezas y oportunidades) comparando “*FitTrack*” y “*Hevy*”:

Fortalezas

- ◆ Funcionalidades complementarias
- ◆ Dinámica de grupos
- ◆ Gratuito para el uso normal

Debilidades

- ◆ Se deben conseguir usuarios
- ◆ Será incompleta al principio
- ◆ Diseño no muy estético

Oportunidades

- ◆ Aspectos del sector sin explotar
- ◆ Grupos y profesionales como público
- ◆ Mucha más conexión con la comunidad

Amenazas

- ◆ Fiabilidad en el mercado
- ◆ Ya funcional y completa
- ◆ Gran comunidad ya establecida

Caso de estudio.

La funcionalidad principal serán las rutinas por lo que la mayor parte de la aplicación girará entorno a esto. Con una vista que tiene un “*navigator*” en la parte inferior y basado en fragmentos, el usuario verá como una de las pestañas el apartado “*Home*”. En este apartado el usuario podrá ver un listado de sus rutinas pudiendo editarlas y añadir más, así como ver detalles de estas.

Dentro de una rutina el usuario podrá añadir ejercicios o eliminar los que tiene asociados. Al añadir un ejercicio podrá buscar en todo el repertorio de ejercicios existentes en la aplicación y públicos en la comunidad para añadir el que más se adecúe al ejercicio que va a realizar. En caso de no existir el ejercicio que busca, lo podrá crear. Si un gran número de personas crea ejercicios parecidos, se añadirá al conjunto de ejercicios global para que cualquier usuario los puedan añadir

Otra de las opciones llevará al usuario a la vista de herramientas. Esta vista mostrará con cómodos iconos una serie de funcionalidades extra que el usuario podrá seleccionar. Al seleccionarlás se llevará al usuario a una actividad donde tendrá lo necesario para utilizar dicha herramienta. En algunos casos la herramienta puede ser cargada previamente con la información que el usuario haya establecido en su perfil.

La última opción es el perfil. Este será el comienzo del módulo social de la aplicación. Por el momento el usuario puede cambiar una serie de datos relacionados con su perfil para que en un futuro el resto de usuarios le puedan identificar e interactuar con él, así como ver sus rutinas y resultados si el usuario quisiese.

Utilizando los roles definidos en la aplicación de nuestro “*backend*” los usuarios podrán ser normales, administradores (solamente trabajadores de la empresa) y, en un futuro, entrenadores. Utilizando la mecánica social de una forma ligeramente adaptada a grupos, los entrenadores podrán tener usuarios a su cargo pudiendo modificar sus rutinas y más.

Diagramas.

En este punto se plasmarán todos los diagramas estructurales de la aplicación para poder observar el funcionamiento de las clases de forma esquemática. Debido al gran tamaño de estos diagramas, serán divididos, agrupados y explicados por separado.

FitTrackApi

En la primera imagen se muestran las clases controladoras. Estas clases se encargan de gestionar las peticiones que llegan al “API”.

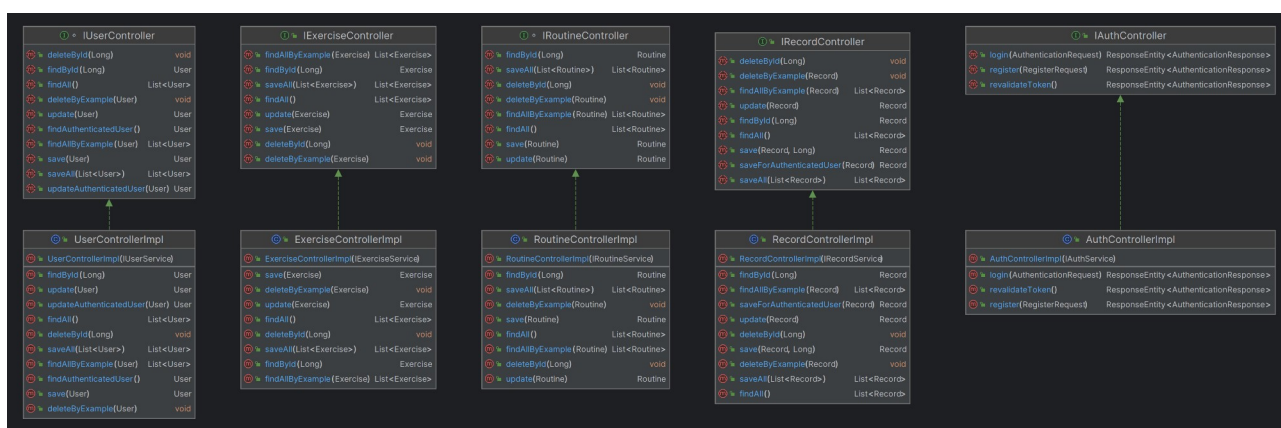


Figura 1: Diagrama de clase de controladores

Estas clases controladoras se encargan de transmitir los datos recibidos de la petición y llamar a los respectivos métodos de los servicios. Estos servicios aplicarán la lógica de negocio pertinente.

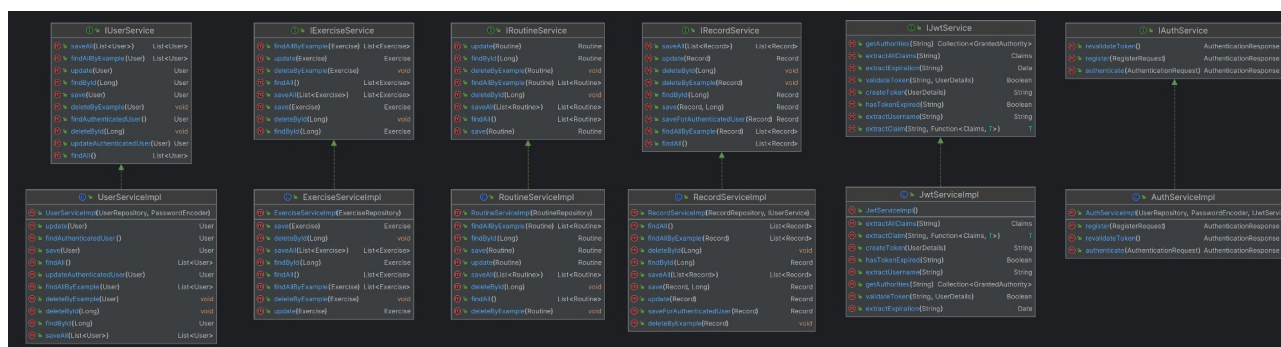


Figura 2: Diagrama de clase de servicios

Para el tratamiento de datos y mapeado de base de datos, se crearon unos modelos. Estos modelos reflejan los datos a tratar para la aplicación y están configurados con los parámetros necesarios para el tratamiento de cada objeto en el entorno al que están destinados, el entorno deportivo.

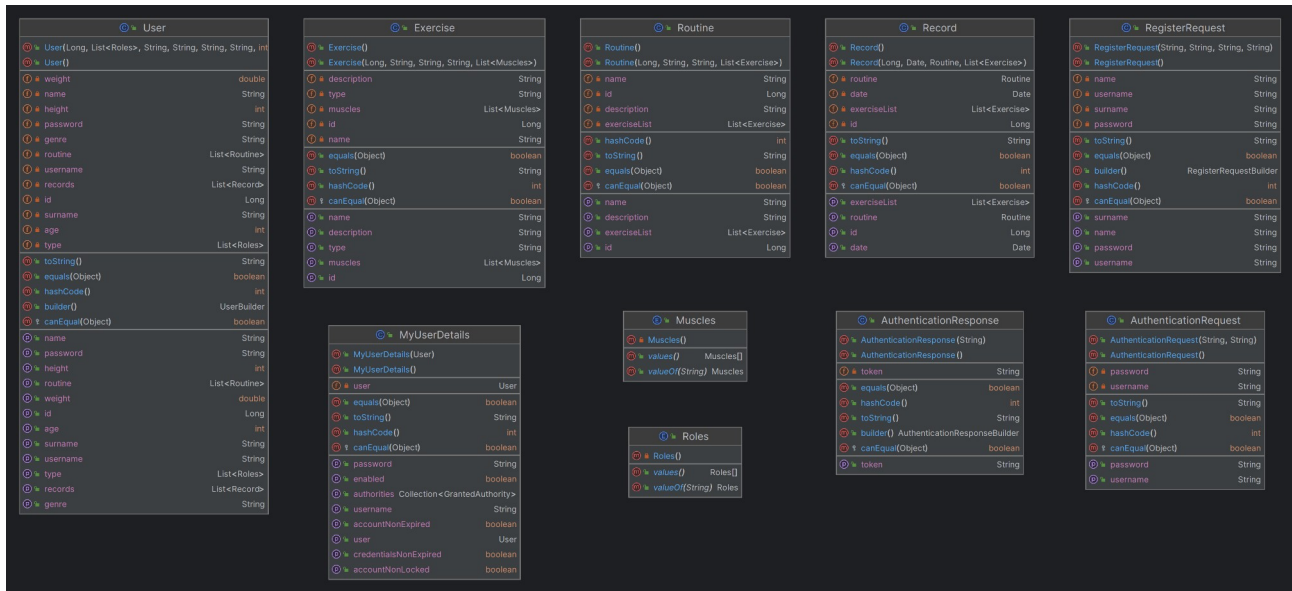


Figura 3: Diagrama de clase de entidades del API

Por último estos servicios llaman a los repositorios. Estos repositorio se encargan de las transacciones con la base de datos para escribir y leer los datos que los servicios hayan requerido. Estas clases son interfaces que implementa “Spring” para facilitar su uso y configuración y asegurar un funcionamiento estándar. En esta imagen se encuentran además diversas clases encargadas del filtrado de peticiones, configuración de seguridad y la propia clase lanzadora de la aplicación.

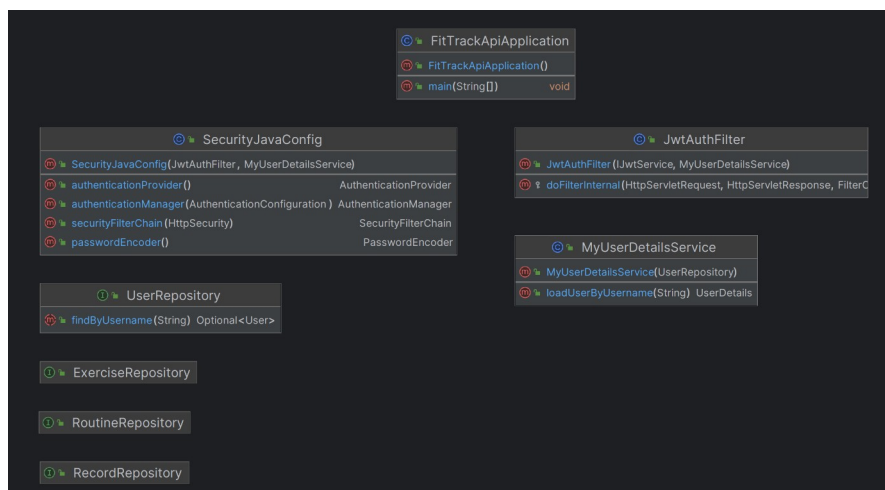


Figura 4: Diagrama de clase de demás clases del API

FitTrack Mobile

Los modelos en el caso de la aplicación móvil no distan mucho del caso de la “Api”. Al ser un proyecto orientado a la persistencia de los datos, no fue necesario agregar lógica asociada a cada entidad. Se han definido los mismos modelos explicados anteriormente para evitar problemas a la hora de guardar la información.



Figura 5: Diagrama de clase de entidades de la aplicación móvil

Las siguientes clases corresponden a todas las vistas de la aplicación y la actividad principal “*Main Activity*”, la cual inicia toda la aplicación. Cada funcionalidad se explicará más adelante. En la parte inferior se ubican las actividades correspondientes a “*Tools*”. Las que terminan en “*Fragment*” son las actividades sobre las que navegaremos en la actividad principal.

FitTrack – Aplicación de seguimiento deportivo.

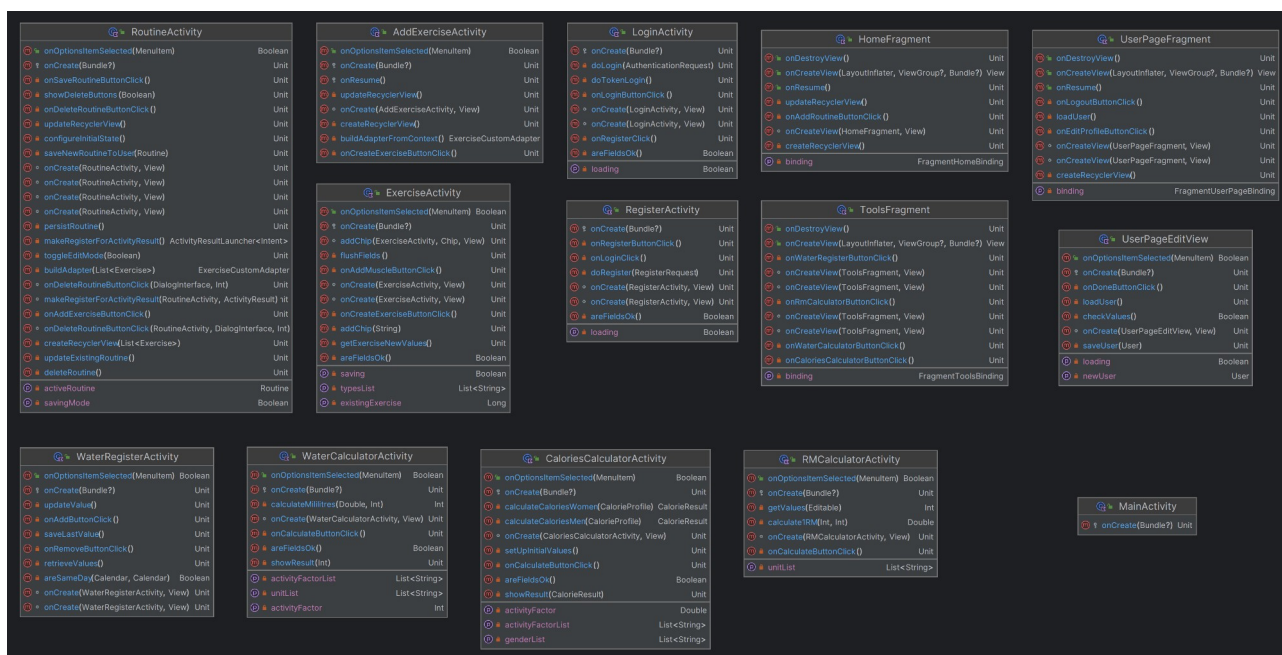


Figura 6: Diagrama de clase de clases asociadas con vistas

Por último, las clases que corresponden con los “*helpers*”, encargados de ejecutar funcionalidades necesitadas de forma repetida a lo largo de la aplicación, los adaptadores, para mostrar las listas, los errores personalizados y las clases encargadas de las peticiones “*Http*”. “*User Context*” se encarga de gestionar los datos del usuario activo, solicitando los datos a “*ApiManager*”, el cual esta configurado con “*HttpClient*” para añadir el “*token*” de autenticación en cada petición.

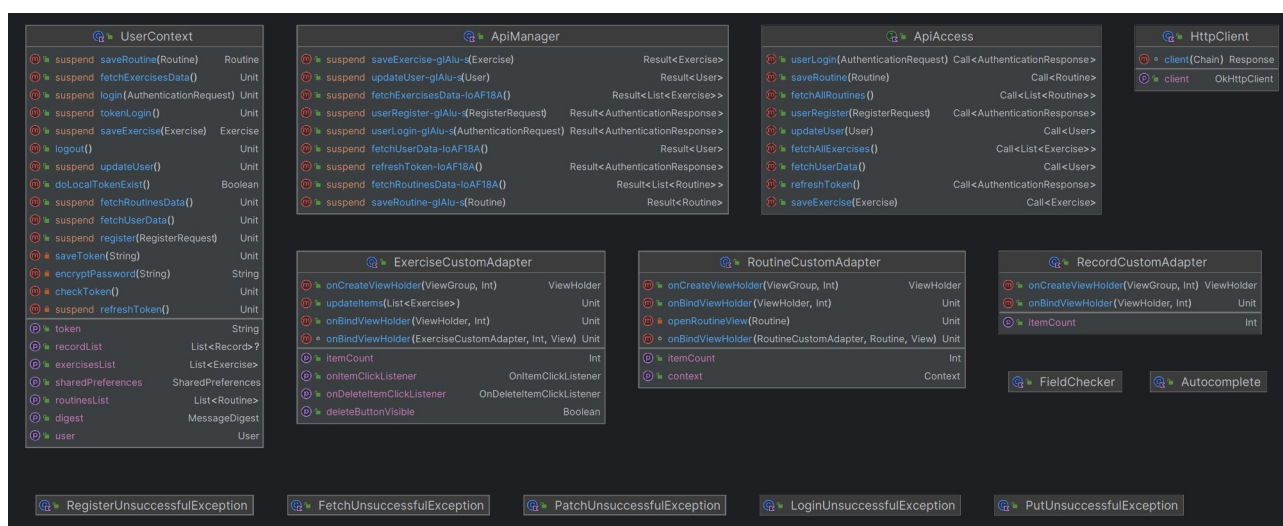


Figura 7: Diagrama de clase de demás clases de la aplicación móvil

Base de datos.

La siguiente imagen muestra el diagrama E-R (Entidad-Relación) de la base de datos. Así como esta mapeada cada una de las clases usando la notación de JPA (Java Persistence API), lo está la base de datos. En este caso se observa que el usuario es la entidad principal. Esta entidad tiene relaciones con “*Records*”, con los tipos de usuario que corresponden los roles, con las rutinas que, así como los “*Records*”, tienen un campo usuario ya que siempre están asociadas a uno, teniendo también el usuario una lista de rutinas y “*Records*” asociados. También se observan las relaciones de los ejercicios con los músculos, con los “*Records*” y con las rutinas. La base de datos esta configurada de tal forma que el usuario sea siempre necesario y que la mayoría de entidades den datos sobre este.

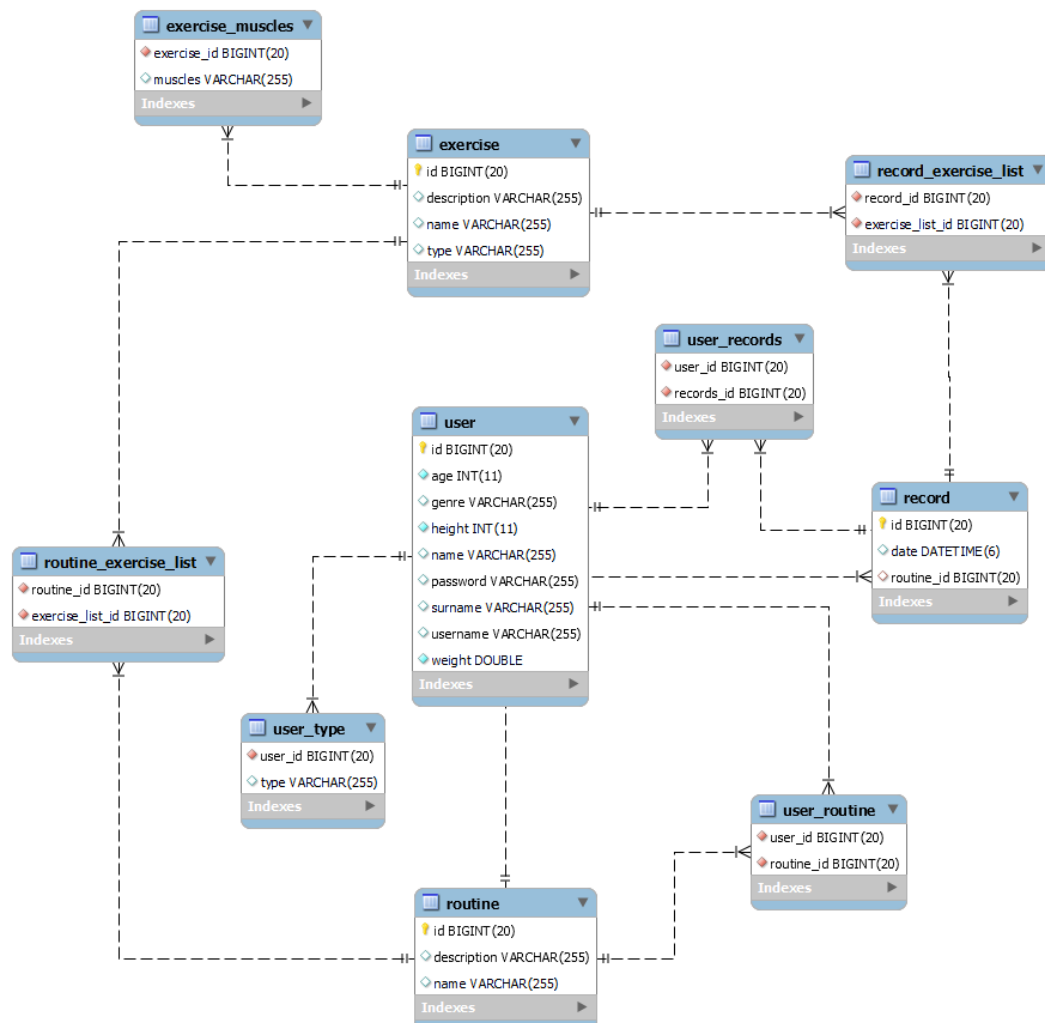


Figura 8: Diagrama entidad-relación de la base de datos

Caso de uso

En el siguiente diagrama se indica el transcurso habitual de un usuario a través de la aplicación. Primeramente se realiza un “login”, manual o automático. El resto de operaciones solo se pueden realizar tras una autenticación exitosa. También se puede observar un segundo actor administrador. Este actor opera sin usar la aplicación móvil. Este actor puede realizar cualquier operación posible sobre cualquier dato de la base de datos a través de los “endpoints” específicos.

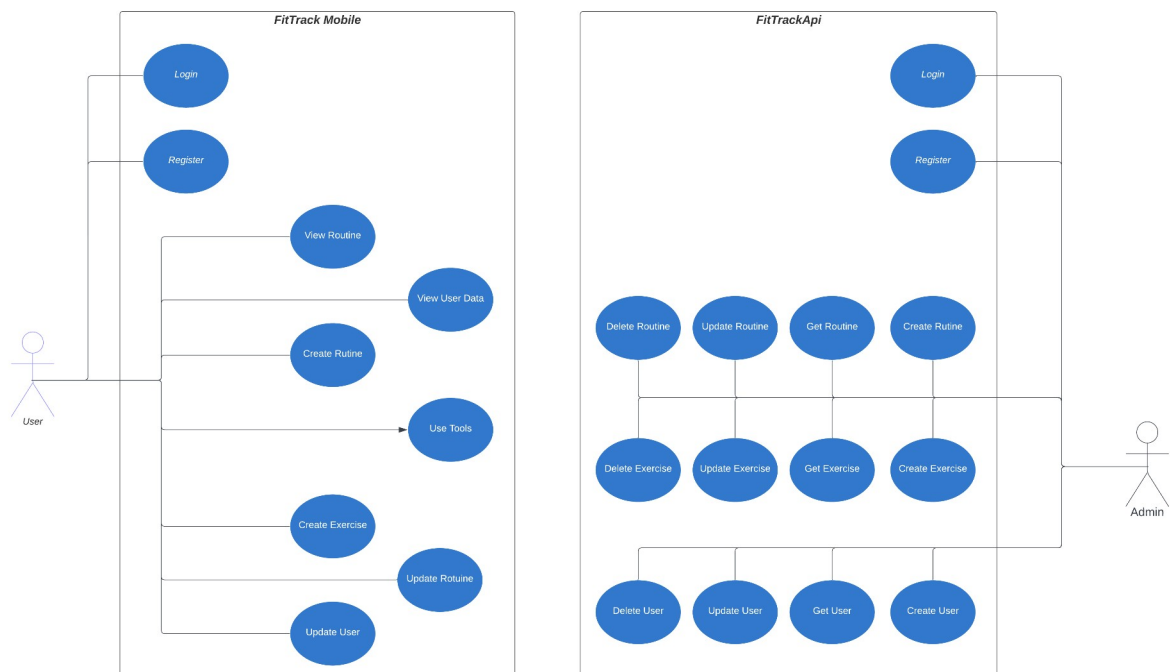


Figura 9: Diagrama de caso de uso

Desarrollo del proyecto

Para un exhaustivo y preciso desarrollo del proyecto, se recorrerá cada una de las pantallas y funcionalidades de la aplicación siguiendo el trayecto de un usuario normal de la aplicación así como las posibilidades del “*Api Rest*”.

FitTrack Mobile

Primeramente se encuentra una pantalla de “*login*”. En este “*login*” se requerirá al usuario introducir sus datos de inicio de sesión para seguidamente hacer clic en el botón de “*login*” y entrar en la aplicación. En caso de no estar previamente registrado el “*login*” no funcionará, por lo que en la parte inferior podrá encontrar un texto que al hacer clic le llevará al formulario de registro. En este formulario se le pedirán sus datos y tras rellenar todos los campos, ya que son requeridos en su totalidad, podrá, de la misma forma que en el “*login*”, hacer clic el botón correspondiente y entrar en la aplicación directamente autenticado.

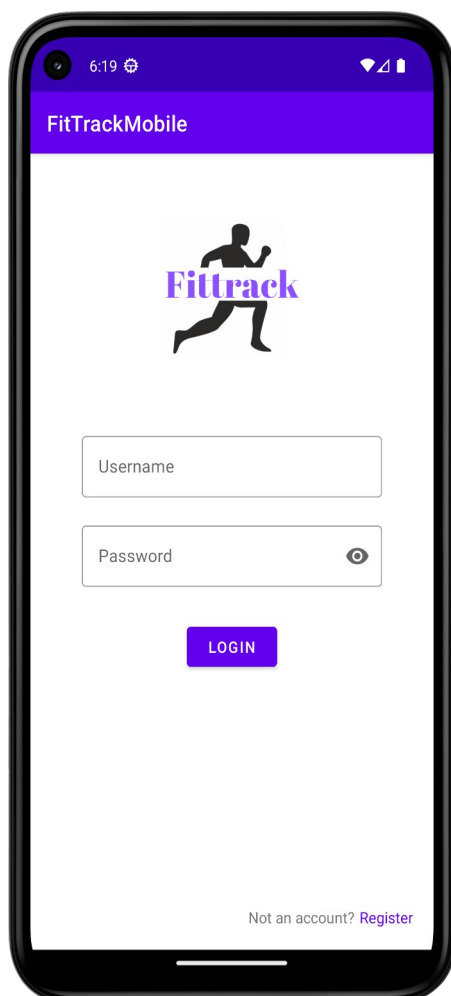


Figura 11: Pantalla de login

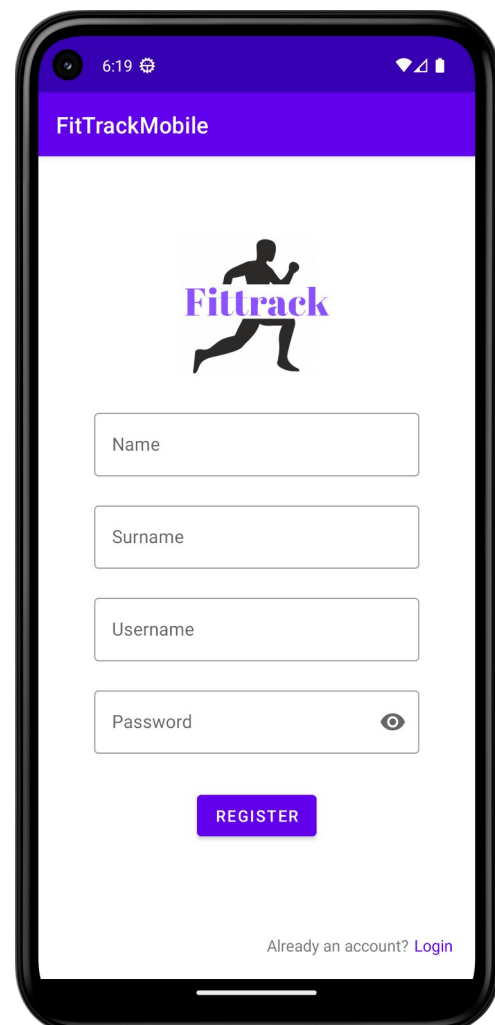


Figura 10: Pantalla de registro

Login Page

Correspondiente a estas vistas, el código encargado de gestionar lo ocurrido en la misma.

En esta clase se encuentra toda la gestión del “login”. Al ser la actividad inicial de la aplicación se utilizará para inicializar, por ejemplo, las “*shared preferences*”, las cuales se utilizarán para obtener el “token” de autenticación y poder entrar directamente a la aplicación sin poner las credenciales. En caso de que este haya caducado el usuario no avanzará hasta haber puesto sus credenciales correctamente. Se pueden observar una serie de “*listeners*” para los botones que aquí se encuentran unos validadores para los campos, las funciones correspondientes tanto al “login” normal como al automático y un método que establecerá el estado de cargando para que el usuario no pueda interactuar.

```
class LoginActivity : AppCompatActivity() {

    private lateinit var binding: ActivityLoginBinding
    private lateinit var userContext: UserContext

    override fun onCreate(savedInstanceState: Bundle?) {
        // ...
    }

    private fun onRegisterClick() {
        startActivity(Intent(this@LoginActivity, RegisterActivity::class.java))
        finish()
    }

    private fun onLoginButtonClick() {
        // ...
    }

    private fun areFieldsOk(): Boolean {
        // ...
    }

    private fun doLogin(authRequest: AuthenticationRequest) {
        // ...
    }

    private fun doTokenLogin() {
        // ...
    }

    private fun setLoading/loading: Boolean) {
        // ...
    }

}
```

Figura 12: Código de pantalla de login

Register Page

El código de la pantalla de registro es bastante similar a la anterior. Tiene los “*listeners*” correspondientes a sus botones con un texto en el que es posible hacer clic, que lleva a la pantalla de “*login*” nuevamente. Sustituyendo al “*login*” tiene el método de registro que hace una llamada a la “*API*” para registrar al usuario y obtener el “*token*” de autenticación. Como en el anterior caso tiene tanto un método para comprobar que todos los campos están rellenos así como un método para establecer la aplicación en un estado de carga impidiendo al usuario realizar interacciones.

```
class RegisterActivity : AppCompatActivity() {

    private lateinit var binding: ActivityRegisterBinding
    private lateinit var userContext: UserContext

    override fun onCreate(savedInstanceState: Bundle?) {
        // ...
    }

    private fun onLoginClick() {
        // ...
    }

    private fun onRegisterButtonClick() {
        // ...
    }

    private fun doRegister(registerRequest: RegisterRequest) {
        // ...
    }

    private fun setLoading/loading: Boolean) {
        // ...
    }

    private fun areFieldsOk(): Boolean {
        // ...
    }
}
```

Figura 13: Código de pantalla de registro

Una vez hecha la autenticación se iniciará la actividad principal de la aplicación. En esta actividad se puede ver en la parte inferior una barra de navegación en la que se encuentran tres iconos. Cada icono corresponderá a un fragmento, el cual brindará una serie de funcionalidades distintas. Ya que cada fragmento tiene la extensión de una actividad, se tratarán por separado.

Home

En esta vista se encuentra una lista de todas las rutinas con las que cuenta el usuario. Las interacciones que podemos realizar aquí consisten en crear rutinas y en acceder a ellas. Para acceder a las rutinas se deberá hacer clic en alguna de ellas, tras ello se abrirá una actividad asociada que se comentará más adelante. Justo después de las rutinas se verá un botón con un aspecto similar a una rutina. Este botón permitirá crear una nueva rutina abriendo una actividad que corresponde con la vista de la rutina, esta se abre al hacer clic en una rutina, pero inicia en el modo de edición directamente.

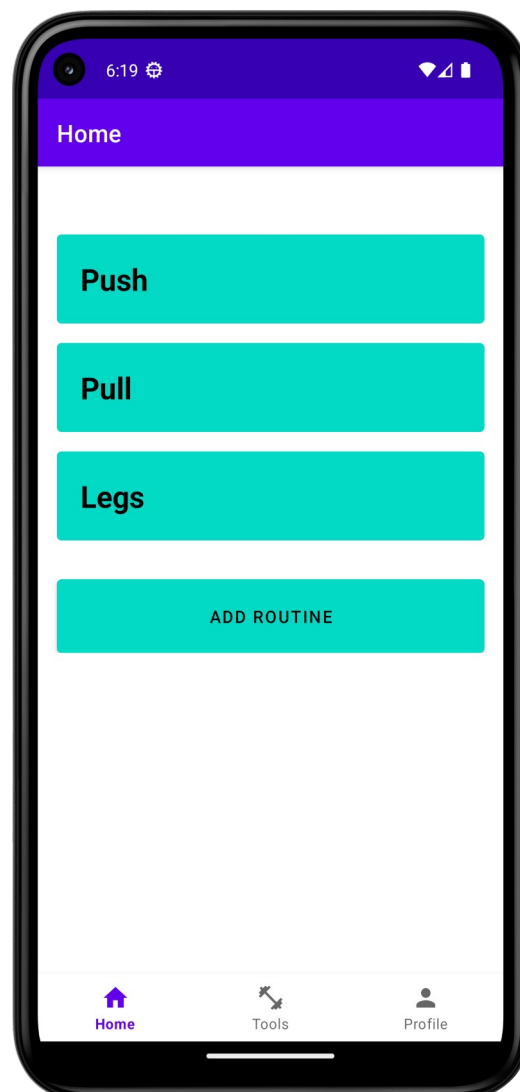


Figura 14: Pantalla de home

El código correspondiente a esta vista es bastante simple. Por una parte están sobrescritos los métodos de “*onResume*” y “*onDestroyView*”, los cuales controlan el comportamiento de la actividad en la reanudación y la destrucción de la misma respectivamente. Se podrá distinguir el “*listener*” del botón mencionado anteriormente, encargado de crear la nueva rutina. Por último se encuentran dos métodos cuyas funciones están relacionadas con el “*recyclerView*”, el cual se encarga de mostrar en forma de lista las rutinas. El “*recyclerView*” cuenta con un adaptador personalizado al que se le ha añadido un método personalizado. Este método se asocia al evento de clic e una rutina, abriéndola en una actividad

```
class HomeFragment : Fragment() {

    private var _binding: FragmentHomeBinding? = null
    private lateinit var userContext: UserContext
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        // ...
    }

    override fun onResume() {
        // ...
    }

    override fun onDestroyView() {
        // ...
    }

    private fun updateRecyclerView() {
        // ...
    }

    private fun onAddRoutineButtonClick() {
        // ...
    }

    private fun createRecyclerView() {
        // ...
    }
}
```

Figura 15: Código de pantalla de home

```
class RoutineCustomAdapter(private val data: Array<Routine>, val context: Context) :
    RecyclerView.Adapter<RoutineCustomAdapter.ViewHolder>() {

    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {

        private val lblTitulo = view.findViewById(R.id.list_item_title) as TextView
        private val lblSubtitulo = view.findViewById(R.id.list_item_description) as TextView

        fun bindItem(routine: Routine){
            // ...
        }
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        // ...
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        // ...
        holder.itemView.setOnClickListener{openRoutineView(routine)}
    }

    override fun getItemCount() = data.size

    private fun openRoutineView(routine: Routine) {
        val intent = Intent(context, RoutineActivity::class.java)
        intent.putExtra("ROUTINEID", routine.id)
        context.startActivity(intent)
    }
}
```

Figura 16: Código de adaptador de rutinas

Tools

En el segundo apartado de la actividad principal se encuentran las “Tools”. En este fragmento se encontrarán cuatro iconos, cada uno correspondiendo una actividad de la aplicación. Estas son:

- *1RM Calculator* – Actividad que permite calcular el peso máximo que el usuario es capaz de levantar en una repetición en base a cuanto ha levantado en cuantas repeticiones.
- *Calories Calculator* – Actividad que en base a al peso, altura, actividad física y género, calcula las calorías basales y de mantenimiento.
- *Water Calculator* – Actividad que en base al peso y la actividad física calcula cuanta agua debería tomar el usuario.
- *Water Register* – Actividad que permite realizar un seguimiento de la cantidad de agua bebida a lo largo del día.

Estas actividades se explicarán al detalle más adelante. El código del fragmento solamente establece el inicio de cada actividad cuando el icono es pulsado

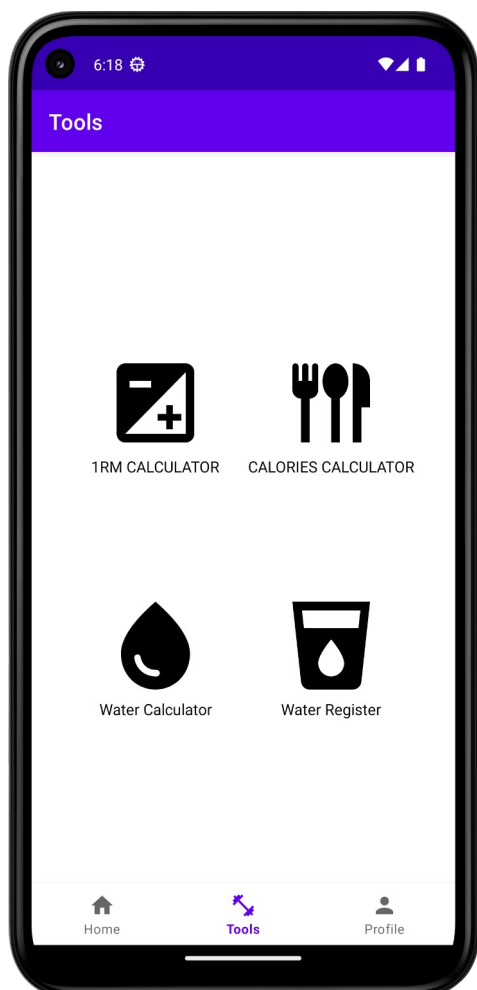


Figura 18: Pantalla de tools

```
class ToolsFragment : Fragment() {

    private var _binding: FragmentToolsBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        // ...
    }

    private fun onWaterCalculatorButtonClick() {
        startActivity(Intent(context, WaterCalculatorActivity::class.java))
    }

    private fun onWaterRegisterButtonClick() {
        startActivity(Intent(context, WaterRegisterActivity::class.java))
    }

    private fun onCaloriesCalculatorButtonClick() {
        startActivity(Intent(context, CaloriesCalculatorActivity::class.java))
    }

    private fun onRmCalculatorButtonClick() {
        startActivity(Intent(context, RmCalculatorActivity::class.java))
    }

    override fun onDestroyView() {
        // ...
    }

}
```

Figura 17: Código de pantalla de tools

Profile

En el fragmento “*Profile*”, el usuario encontrará su información. Aquí se mostrarán datos como el peso o la altura, los cuales pueden ser modificados. Justo debajo del icono de usuario se encuentra el botón de edición y, a su lado, el botón de “*Logout*”. El primero servirá para llevar al usuario a una nueva actividad en la cual podrá modificar estos datos. El segundo marcado en rojo, realizará el proceso de cierre de sesión eliminando el “*token*” de autenticación del usuario, obligando a introducir de nuevo las credenciales o realizar un nuevo registro. En la parte inferior encontramos lo que en un futuro será una lista con los últimos entrenamientos realizados por el usuario.

El código asociado a esta actividad controlará la carga de los datos del usuario así como establecer las acciones a realizar con las pulsaciones de cada uno de los botones.

```
class UserPageFragment : Fragment() {

    private var _binding: FragmentUserPageBinding? = null
    private val binding get() = _binding!!

    private lateinit var userContext: UserContext

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        // ...
    }

    override fun onResume() {
        super.onResume()
        loadUser()
    }

    private fun onLogoutButtonClick() {
        // ...
    }

    private fun LoadUser() {
        // ...
    }

    private fun createRecyclerView() {
        // ...
    }

    private fun onEditProfileButtonClick() {
        // ...
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Figura 20: Código de pantalla del perfil

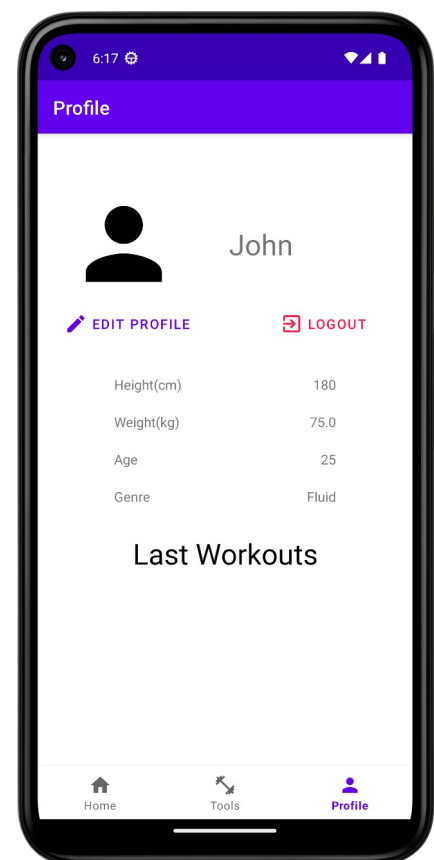


Figura 19: Pantalla del perfil

En este momento se procede a explicar cada una de las actividades que no corresponden a la vista principal. Estas actividades son accesibles a través de los fragmentos de la actividad principal y se indicará a través de que actividad se pueden encontrar. Todas estas actividades contarán con un botón en la barra de acción superior. El funcionamiento de este siempre será regresar a la anterior actividad en caso de no especificar otro uso.

Routine

Esta actividad es accesible desde el fragmento de “Home”, en el que se encontraba un listado con todas las rutinas. Al seleccionar una de ellas se abrirá esta actividad en la que se puede ver el nombre de la rutina en la parte superior además de una lista con los ejercicios asociados. Al lado del nombre se encuentra el botón de edición, identificable por su forma de lápiz. Al pulsar el botón entraremos en el modo edición, que además, es el modo por defecto cuando creamos una nueva rutina.

En la vista de edición se mostrarán una serie de botones nuevos e iconos. Estos botones realizarán acciones sobre la propia rutina mientras que los iconos con forma de papelera permitirán eliminar los ejercicios de la rutina. Al seleccionar el botón de “Add Exercise” se abrirá una actividad que permitirá seleccionar

ejercicios a añadir. El botón de “Save Routine” y “Delete Routine” terminan ambos el proceso de edición, uno persistiendo los cambios en la base de datos y el otro eliminando la rutina de la lista de rutinas del usuario.

En este caso el botón de la barra de acción saldrá del modo de edición.

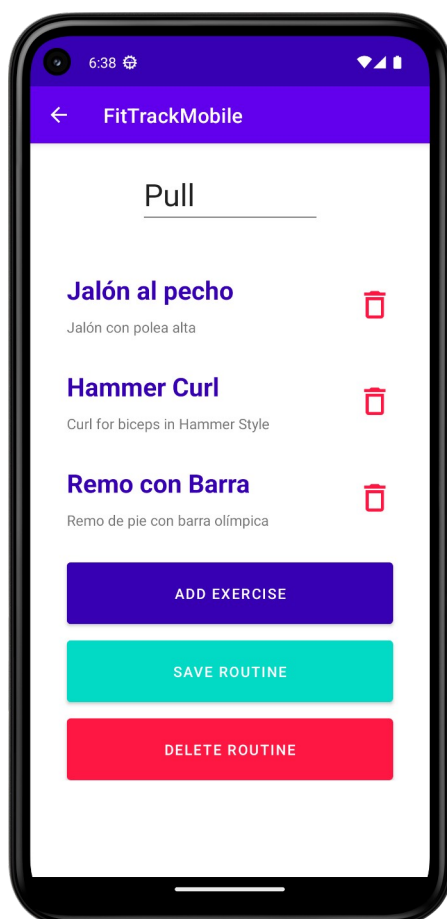


Figura 22: Pantalla de vista de rutina en modo edición

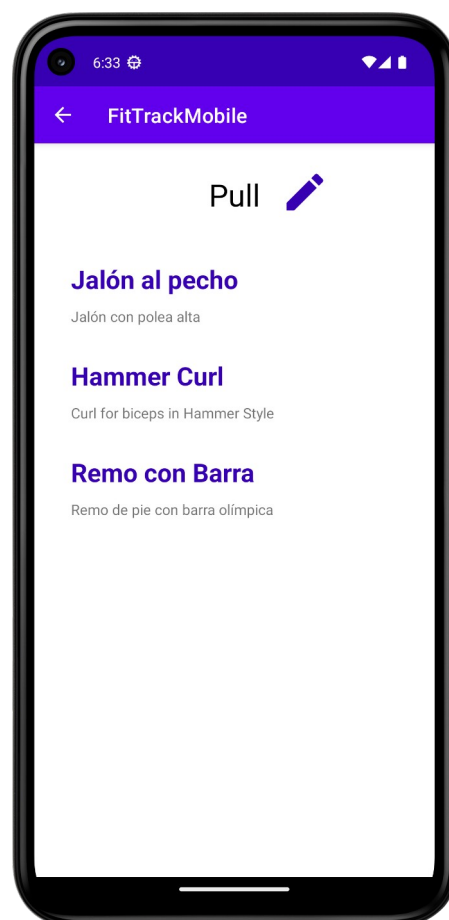


Figura 21: Pantalla de vista de rutina

El código incluye desde la carga de los datos de la rutina hasta controladores de los resultados de otras actividades.

En esta clase convergen diversas funcionalidades que permite realizar el SDK (Software Development Kit) de Android. Por una parte, en el “*listener*” del botón de añadir ejercicio, se lanza una actividad previamente configurada para devolver una respuesta al ser terminada. Con esta respuesta se controla el añadido del ejercicio a la rutina para su futura persistencia o descarte. El botón de eliminar rutina disparará un “*Alert Dialog*” en el que se pedirá una confirmación para la eliminación de la rutina. Después se encuentran los métodos encargados de persistir la rutina de forma distinta en base a si es nueva. Por último, los métodos encargados de mostrar y actualizar el “*RecyclerView*”.

```
class RoutineActivity : AppCompatActivity() {

    private lateinit var binding: ActivityRoutineBinding
    private lateinit var userContext: UserContext
    private lateinit var getExerciseToAdd : ActivityResultLauncher<Intent>
    private lateinit var exerciseAdapter: ExerciseCustomAdapter
    private var routineId = -1L
    private var activeRoutine = Routine()
    private var isNewRoutine = false
    private var isEditing = false

    override fun onCreate(savedInstanceState: Bundle?) {
        // ...
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        // ...
    }

    private fun configureInitialState() {
        // ...
    }

    private fun onDeleteRoutineButtonClick() {
        // ...
    }

    private fun deleteRoutine() {
        // ...
    }

    private fun setActiveRoutine(routine: Routine) {
        // ...
    }

    private fun makeRegisterForActivityResult(): ActivityResultLauncher<Intent> {
        // ...
    }

    // ...
}
```

Figura 23: Código de pantalla de vista de rutina (1/2)

```
class RoutineActivity : AppCompatActivity() {
    // ...

    private fun onAddExerciseButtonClick() {
        // ...
    }

    private fun onSaveRoutineButtonClick() {
        // ...
    }

    private fun updateExistingRoutine() {
        // ...
    }

    private fun persistRoutine() {
        // ...
    }

    private fun saveNewRoutineToUser(newRoutine: Routine) {
        // ...
    }

    private fun setSavingMode(isSaving: Boolean) {
        // ...
    }

    private fun toggleEditMode(inEditMode: Boolean) {
        // ...
    }

    private fun createRecyclerView(exerciseList: MutableList<Exercise>) {
        // ...
    }

    private fun buildAdapter(exerciseList: MutableList<Exercise>): ExerciseCustomAdapter {
        // ...
    }

    private fun updateRecyclerView() {
        // ...
    }

    private fun showDeleteButtons(show: Boolean) {
        // ...
    }
}
```

Figura 24: Código de pantalla de vista de rutina (2/2)

Add Exercise

Al pulsar en el botón “Add Exercise” en la actividad de rutina, mencionada arriba, se abrirá esta actividad. En esta actividad se mostrará una lista con todos los ejercicios que se encuentren en la base de datos de la aplicación. Al hacer clic en uno de ellos se regresará a la actividad anterior y este se añadirá a la rutina que se estuviese modificando en ese momento. En caso de querer crear uno propio, hay un botón en la parte inferior que abrirá la actividad de creación de ejercicio. Tras crearlo se encontrará en esta misma lista.

El código asociado controlará la creación de la lista, definiendo el comportamiento de la aplicación al pulsar en uno de los elementos de esta y el inicio de la actividad de crear un ejercicio al pulsar el botón. También controlará la actualización de la lista.

```
class AddExerciseActivity : AppCompatActivity() {

    private lateinit var binding: ActivityAddExerciseBinding
    private lateinit var userContext: UserContext

    override fun onCreate(savedInstanceState: Bundle?) {
        // ...
    }

    override fun onResume() {
        updateRecyclerView()
        super.onResume()
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        // ...
    }

    private fun onCreateExerciseButtonClick() {
        // ...
    }

    private fun createRecyclerView() {
        // ...
    }

    private fun updateRecyclerView() {
        // ...
    }

    private fun buildAdapterFromContext(): ExerciseCustomAdapter {
        // ...
    }

}
```

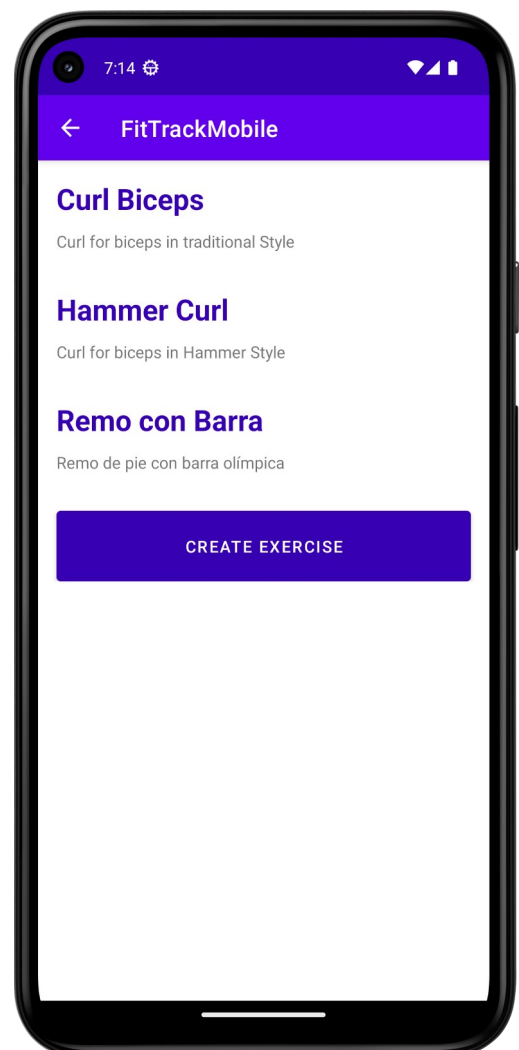


Figura 25: Código de pantalla para añadir ejercicio

Figura 26: Pantalla para añadir ejercicio

Create Exercise

Al seleccionar “*Create Exercise*” en la actividad anterior se abrirá esta. En esta se podrá ver un simple formulario de creación en el que se indicará una serie de datos del ejercicio como pueden ser, los músculos implicados o una descripción de este. Uno de los campos desplegables tendrá al lado un botón con el texto “*Add*”, este botón añadirá el músculo seleccionado a un “*Chip Group*” en el que se podrá ver el nombre de cada músculo junto a un botón para eliminarlo de la agrupación. Al pulsar el botón de “*Create Exercise*” se guardará el ejercicio en la base de datos y se volverá a la lista de ejercicios, donde ahora se encontrará el ejercicio creado.

El código asociado controlará el comportamiento de los botones y el establecimiento de un estado de carga para evitar interacciones del usuario. Al añadir un músculo controlará que no este añadido ya, y se comprobará que los campos necesarios estén cubiertos.

```
class ExerciseActivity : AppCompatActivity() {

    private lateinit var binding: ActivityExerciseBinding
    private lateinit var userContext: UserContext
    private lateinit var chipGroup: ChipGroup
    private val selectedMuscles = mutableListOf<Muscles>()
    private var exercise = Exercise()

    override fun onCreate(savedInstanceState: Bundle?) {
        // ...
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        // ...
    }

    private fun setExistingExercise(exerciseId: Long) {
        // ...
    }

    private fun flushFields() {
        // ...
    }

    private fun getExerciseNewValues() {
        // ...
    }

    private fun onCreateExerciseButtonClick() {
        // ...
    }

    private fun areFieldsOk(): Boolean {
        // ...
    }

    private fun onAddMuscleButtonClick() {
        // ...
    }

    private fun addChip(muscle: String) {
        // ...
    }

    private fun setSaving(isSaving: Boolean) {
        // ...
    }

    private fun getTypesList(): List<String> {
        // ...
    }
}
```

Figura 28: Código de pantalla de creación de ejercicio

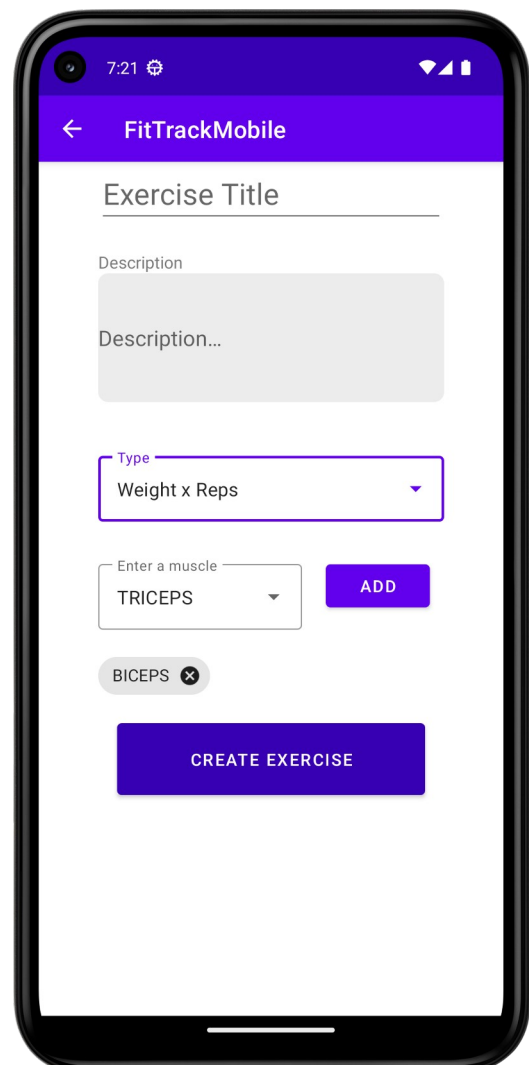


Figura 27: Pantalla de creación de ejercicio

User Edit Page

Volviendo a la actividad principal, en el fragmento de usuario se encontraba un botón de edición, al pulsar ese botón se inicia esta actividad. Esta actividad consta de unos campos de texto en los que se podrá editar la información básica del usuario. En la parte superior se mostrará el nombre del usuario. Esto no se debe confundir con el “username” ya que este no es modificable una vez creada la cuenta. En la parte inferior se encuentra el botón “Done”, que persistirá nuestros datos en la base de datos.

El código controlará las acciones de llamar al guardado y de comprobación de los campos, además de cargar el estado inicial y gestionar el período de guardado.

```
class UserPageEditView : AppCompatActivity() {

    private lateinit var binding: ActivityUserPageEditViewBinding

    private lateinit var userContext: UserContext

    private lateinit var newName: EditText
    private lateinit var newWeight: EditText
    private lateinit var newHeight: EditText
    private lateinit var newAge: EditText
    private lateinit var newGenre: EditText

    override fun onCreate(savedInstanceState: Bundle?) {
        // ...
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        // ...
    }

    private fun onDoneButtonClick() {
        // ...
    }

    private fun getNewUser(): User {
        // ...
    }

    private fun saveUser(newUser: User) {
        // ...
    }

    private fun checkValues(): Boolean {
        // ...
    }

    private fun loadUser() {
        // ...
    }

    private fun setLoading(isLoading: Boolean) {
        // ...
    }

}
```

Figura 29: Código de pantalla de edición del perfil

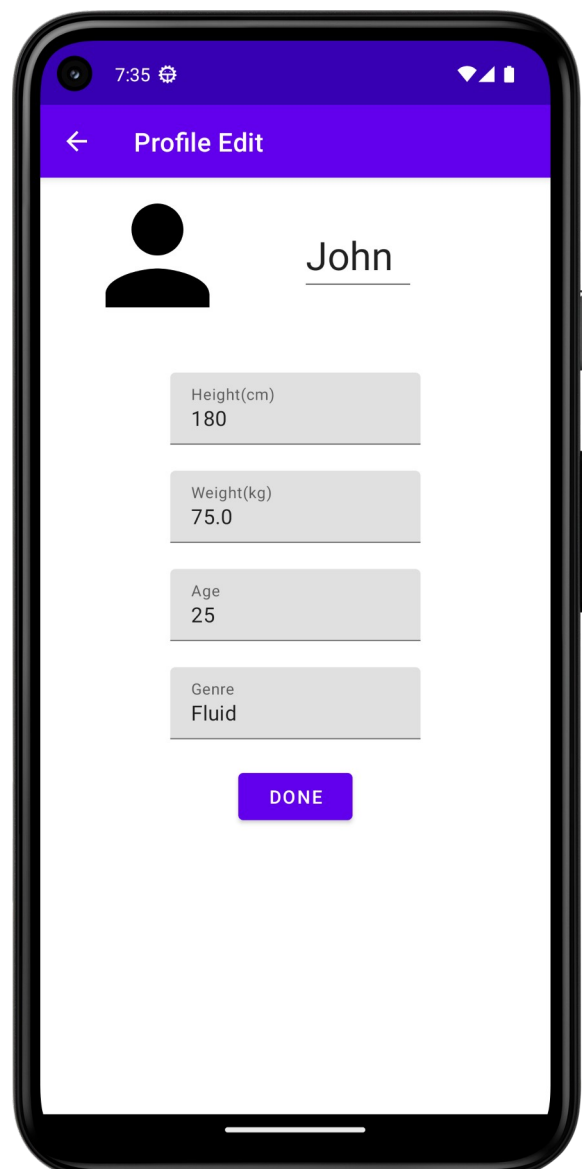


Figura 30: Pantalla de edición del perfil

Para terminar con las vistas, se mostrarán y explicarán las herramientas que se encuentran en el fragmento “tools” de la aplicación. Estas actividades buscan cumplir un papel en el desarrollo de una persona deportista.

1RM Calculator

Esta actividad consiste en la aplicación de una fórmula muy aceptada por la comunidad para calcular el 1RM de un ejercicio. Esto consiste en el peso máximo que se puede levantar en el ejercicio. Para calcular eso se necesita el peso actual levantado con el número de repeticiones.

Para conseguir este objetivo, se podrán observar dos campos de texto y un desplegable. Al introducir los datos pedidos en cada campo y pulsar el botón “Calculate”, se mostrará el resultado de la operación.

El código consta de una comprobación simple de los campos y el propio cálculo.

```
class RMCalculatorActivity : AppCompatActivity() {  
  
    private lateinit var binding : ActivityRmcalculatorBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        // ...  
    }  
  
    override fun onOptionsItemSelected(item: MenuItem): Boolean {  
        // ...  
    }  
  
    private fun onCalculateButtonClick() {  
        // ...  
    }  
  
    private fun calculate1RM(weight: Int, reps: Int): Double {  
        // ...  
    }  
  
    private fun getValues(field : Editable?): Int {  
        // ...  
    }  
  
    private fun getUnitList(): List<String> {  
        // ...  
    }  
}
```

Figura 32: Código de pantalla de calculadora de "RM"

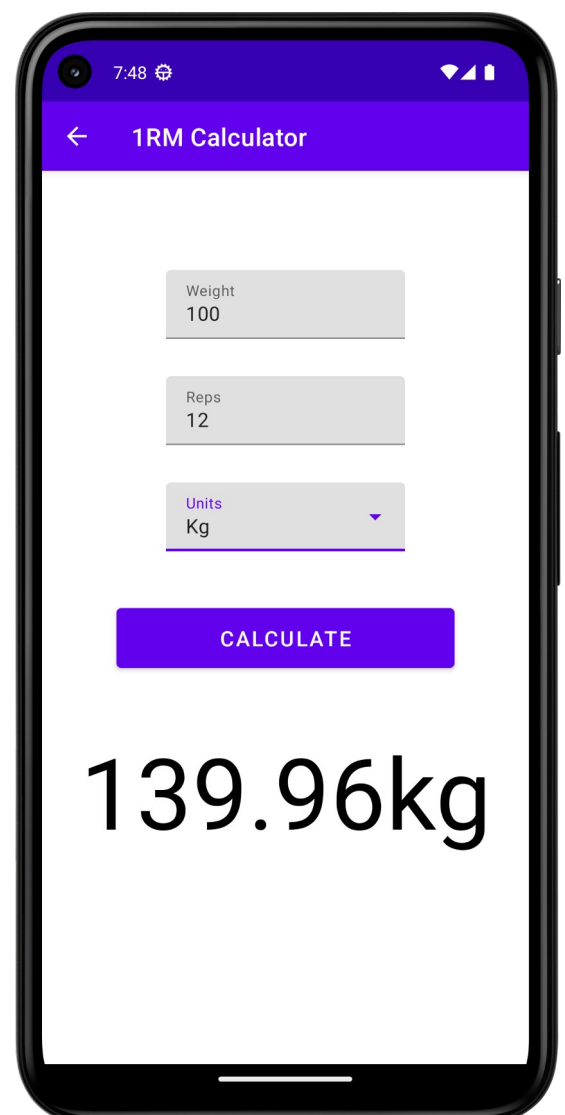


Figura 31: Pantalla de calculadora de "RM"

Calories Calculator

Es muy importante para conseguir un buen rendimiento deportivo tener un seguimiento de la alimentación. Para esto es básico tener conocimiento sobre las calorías que tu cuerpo necesita para subsistir y actuar en consecuencia. Con una serie de datos básicos como es peso, altura, sexo, edad y actividad física, se puede calcular esta cifra.

Esta es la utilidad de la siguiente actividad. Al iniciar la actividad se introducirán de forma automática los datos del usuario “*loggeado*”. Al completar el resto de campos y pulsar el botón “*Calculate*” se mostrará el resultado tanto de las calorías basales (calorías que tu cuerpo necesita sin tener en cuenta el ejercicio) y las de mantenimiento (calorías que tu cuerpo necesita para mantener el peso actual).

El código controla la carga de los datos del usuario en los respectivos campos, el cálculo del resultado y la muestra de este. Además aquí se definen dos “*data class*” que se utilizarán para la gestión de los datos dentro de la actividad.

```
class CaloriesCalculatorActivity : AppCompatActivity() {
    private lateinit var binding: ActivityCaloriesCalculatorBinding
    private lateinit var userContext: UserContext

    override fun onCreate(savedInstanceState: Bundle?) {
        // ...
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        // ...
    }

    private fun setUpInitialValues() {
        // ...
    }

    private fun onCalculateButtonClick() {
        // ...
    }

    private fun showResult(result: CalorieResult) {
        // ...
    }

    private fun getActivityFactor(): Double {
        // ...
    }

    private fun calculateCaloriesWomen(profile: CalorieProfile): CalorieResult {
        // ...
    }

    private fun calculateCaloriesMen(profile: CalorieProfile): CalorieResult {
        // ...
    }

    private fun areFieldsOk(): Boolean {
        // ...
    }

    private fun getActivityFactorList(): List<String> {
        // ...
    }

    private fun getGenderList(): List<String> {
        // ...
    }

    data class CalorieProfile(
        val weight: Double,
        val height: Int,
        val age: Int,
        val activityFactor: Double
    )

    class CalorieResult(val basalCalories: String, val maintenanceCalories: String) {
        constructor(basalCalories: Int, maintenanceCalories: Int) : this(
            "$basalCalories kcal", "$maintenanceCalories kcal"
        )
    }
}
```

Figura 34: Código de pantalla de calculadora de calorías

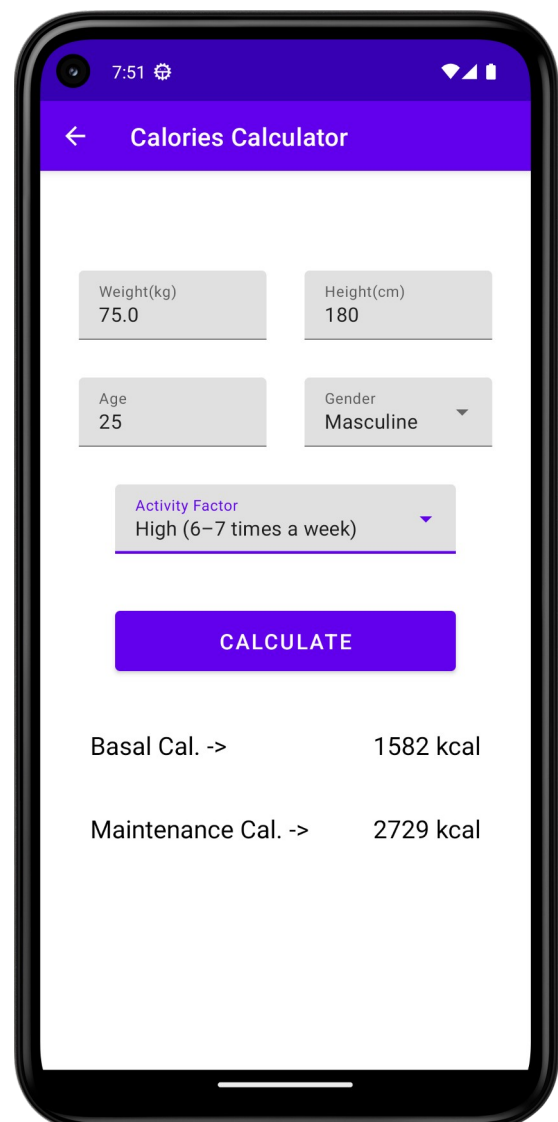


Figura 33: Pantalla de calculadora de calorías

Water Calculator

La actividad de “*Water Calculator*” permitirá calcular el agua que necesita tomar una persona en base a su peso y a su actividad física. La cifra calculada en esta actividad es una estimación pero puede dar una buena referencia de lo mínimo que una persona debería tomar en su día a día.

Para realizar el cálculo se deberán rellenar todos los campos. Estos consistirán en un campo de texto y dos desplegables. Tras introducir el peso, el cual vendrá previamente introducido, el factor de actividad, las unidades de cálculo y pulsar el botón, se mostrará el resultado en la parte inferior en la unidad indicada.

El código, como en anteriores casos, controlará el cálculo, la obtención de los datos y la muestra del resultado.

```
class WaterCalculatorActivity : AppCompatActivity() {

    private lateinit var binding: ActivityWaterCalculatorBinding
    private lateinit var userContext: UserContext
    private val CONVERSION_VALUE = 0.033814

    override fun onCreate(savedInstanceState: Bundle?) {
        // ...
    }

    private fun onCalculateButtonClick() {
        // ...
    }

    private fun showResult(result: Int) {
        // ...
    }

    private fun getActivityFactor(): Int {
        // ...
    }

    private fun areFieldsOk(): Boolean {
        // ...
    }

    private fun calculateMililitres(weight: Double, activityFactor: Int): Int {
        // ...
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        // ...
    }

    private fun getActivityFactorList(): List<String> {
        // ...
    }

    private fun getUnitList(): List<String> {
        // ...
    }
}
```

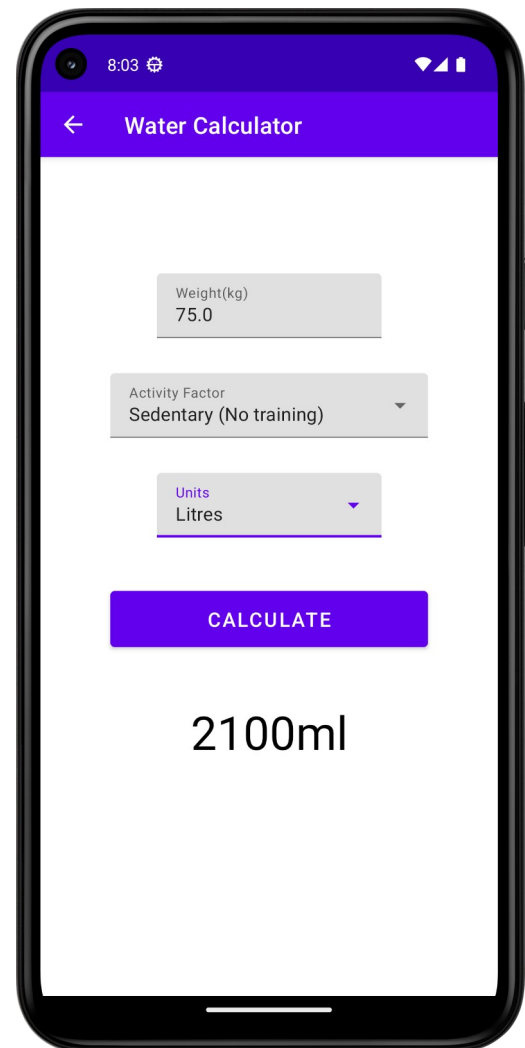


Figura 36: Pantalla de calculadora de agua

Figura 35: Código de pantalla de calculadora de agua

Water Register

Tomar agua es muy importante para llevar una vida saludable y recuperar los líquidos perdidos a lo largo del día y el entrenamiento. Esta actividad permitirá realizar un seguimiento dentro de un mismo día. El funcionamiento de esta actividad se limita a dos botones. Uno de ellos sube la cantidad de vasos de agua tomados, el otro la reduce, teniendo como límite el cero.

El código de la aplicación controla los eventos de pulsado de los botones y guarda la información en el almacenamiento del dispositivo. También se encarga de obtener el último valor y reiniciarlo si el último vaso añadido tiene como fecha el día anterior.

```
class WaterRegisterActivity : AppCompatActivity() {  
  
    private lateinit var binding: ActivityWaterRegisterBinding  
    private lateinit var sharedPreferences: SharedPreferences  
    private var waterValue = 0  
    private var lastGlass = 1L  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        // ...  
    }  
  
    override fun onOptionsItemSelected(item: MenuItem): Boolean {  
        // ...  
    }  
  
    private fun onRemoveButtonClick() {  
        // ...  
    }  
  
    private fun onAddButtonClick() {  
        // ...  
    }  
  
    private fun updateValue() {  
        // ...  
    }  
  
    private fun saveLastValue() {  
        // ...  
    }  
  
    private fun retrieveValues() {  
        // ...  
    }  
  
    private fun areSameDay(calendar1: Calendar, calendar2: Calendar): Boolean {  
        // ...  
    }  
}
```

Figura 37: Código de pantalla de registro de agua



Figura 38: Pantalla de registro de agua

Una vez terminadas todas las vistas de la aplicación, resta por el resto del funcionamiento que no tiene una vista asociada. Se excluirán las clases más básicas como las entidades, excepciones, adaptadores o los “*helpers*”.

Context

Para el manejo del estado del usuario se ha seguido un patrón similar al que tienen “*frameworks*” como “*React*” para la gestión de un contexto. El funcionamiento consiste en una clase con el patrón “*singleton*”. Esta clase esta directamente conectada con otra que realiza todas las peticiones a la “*API*”. En esta clase se encuentran métodos encargados de obtener los datos del usuario autenticado, modificarlos, obtener datos de rutinas y ejercicios, realizar la autenticación, el “*logout*”, entre otros.

```
class UserContext: ViewModel() {

    lateinit var sharedPreferences: SharedPreferences
    var user: User = User()
    var recordList: List<Record>? = null
    private val apiManager = ApiManager.getInstance()
    var token: String = ""
    private var tokenCreation = Date()
    var exercisesList: List<Exercise> = ArrayList()
    var routinesList: List<Routine> = ArrayList()
    var digest: MessageDigest = MessageDigest.getInstance("SHA-256")

    companion object {
        @Volatile
        private var instance: UserContext? = null

        fun getInstance(): UserContext {
            return instance ?: synchronized(this) {
                instance ?: UserContext().also { instance = it }
            }
        }
    }

    suspend fun updateUser() {
        // ...
    }

    suspend fun fetchUserData() {
        // ...
    }

    suspend fun fetchExercisesData() {
        // ...
    }

    suspend fun fetchRoutinesData() {
        // ...
    }

    suspend fun saveRoutine(routine: Routine): Routine {
        // ...
    }

    suspend fun Login(authenticationRequest: AuthenticationRequest) {
        // ...
    }
}
```

Figura 39: Código del contexto de usuario (1/2)

```
class UserContext: ViewModel() {
    // ...

    suspend fun register(registerRequest: RegisterRequest) {
        // ...
    }

    suspend fun saveExercise(exercise: Exercise): Exercise {
        // ...
    }

    private suspend fun refreshToken() {
        // ...
    }

    suspend fun tokenLogin() {
        // ...
    }

    @OptIn(DelicateCoroutinesApi::class)
    private fun checkToken() {
        // ...
    }

    private fun saveToken(newToken: String) {
        // ...
    }

    private fun encryptPassword(password: String): String {
        // ...
    }

    fun doLocalTokenExist(): Boolean {
        // ...
    }

    fun Logout() {
        // ...
    }
}
```

Figura 40: Código del contexto de usuario (2/2)

Api Manager

Para las peticiones se utiliza el paquete “*retrofit*”, el cual a través de la definición de una interfaz con notaciones propias del paquete, generar peticiones base. Para la realización de las peticiones se utilizaron corutinas, asegurando la obtención de los datos en el momento necesario. Todo esto se aplica en esta clase. La cual de forma similar realiza todas las peticiones correspondientes a través de la interfaz “*Api Access*”. Esta clase también utiliza el patrón “*singleton*”.

Para configurar “*retrofit*” se utiliza el “*builder*” que viene incorporado en la clase “*Retrofit*”. El resto de métodos corresponden a cada llamada al “*Api*”.

```
class ApiManager {

    companion object {
        @Volatile
        private var instance: ApiManager? = null

        fun getInstance(): ApiManager {
            return instance ?: synchronized(this) {
                instance ?: ApiManager().also { instance = it }
            }
        }
    }

    private val retrofit: Retrofit = Retrofit.Builder()
        .baseUrl("http://144.24.197.53:8080")
        .client(HttpClient().client)
        .addConverterFactory(GsonConverterFactory.create())
        .build()

    private val apiAccess: ApiAccess = retrofit.create(ApiAccess::class.java)

    suspend fun userLogin(authenticationRequest: AuthenticationRequest): Result<AuthenticationResponse?> {
        // ...
    }

    suspend fun userRegister(registerRequest: RegisterRequest): Result<AuthenticationResponse?> {
        // ...
    }

    suspend fun fetchUserData(): Result<User?>{
        // ...
    }

    suspend fun updateUser(user: User): Result<User?>{
        // ...
    }

    suspend fun refreshToken(): Result<AuthenticationResponse?> {
        // ...
    }

    suspend fun fetchExercisesData(): Result<List<Exercise>> {
        // ...
    }

    suspend fun fetchRoutinesData(): Result<List<Routine>> {
        // ...
    }

    suspend fun saveRoutine(routine: Routine): Result<Routine?> {
        // ...
    }

    suspend fun saveExercise(exercise: Exercise): Result<Exercise?>
        // ...
    }
}
```

Figura 42: Código de "Api Manager"

```
interface ApiAccess {

    @POST("/auth/login")
    fun userLogin(@Body authRequest: AuthenticationRequest): Call<AuthenticationResponse>

    @POST("/auth/register")
    fun userRegister(@Body registerRequest: RegisterRequest): Call<AuthenticationResponse>

    @PATCH("/users/myUser")
    fun updateUser(@Body user: User): Call<User>

    @GET("/users/myUser")
    fun fetchUserData(): Call<User>

    @GET("/auth/revalidate")
    fun refreshToken(): Call<AuthenticationResponse>

    @GET("/exercises")
    fun fetchALLExercises(): Call<List<Exercise>>

    @GET("/routines")
    fun fetchALLRoutines(): Call<List<Routine>>

    @POST("/routines")
    fun saveRoutine(@Body routine: Routine): Call<Routine>

    @POST("/exercises")
    fun saveExercise(@Body exercise: Exercise): Call<Exercise>
}
```

Figura 41: Código de "Api Access"

API

Para un funcionamiento completo de la aplicación se ha desarrollado un “API” completamente funcional y estable, a través de la cual se pueden persistir todos los datos requeridos. La creación de esta “API” está desligada de la aplicación de Android, significando esto que sus funcionalidades no están exclusivamente diseñadas para el uso de la aplicación móvil si no también para futuros desarrollos. Se encontrará la aplicación dividida en:

Controladores – Encargados de gestionar las peticiones “Http”.

Servicios – Encargados de definir y aplicar la lógica de negocio

Repositorios – Encargados de realizar la conexión a la base de datos.

Entidades – Clases que representan las tablas en la base de datos

En este desarrollo se han seguido diversas buenas prácticas con el fin de crear un código limpio y sostenible.

Auth

El primer controlador esencial para la aplicación móvil es el de autenticación. Este controlador permitirá obtener los “tokens” de autenticación para hacer el resto de acciones posibles en la “API”.

Aquí se encuentran los siguientes “endpoints”:

Login – Manejará la llegada de un usuario y contraseña y llamará a la validación

Regsiter – Con los datos del nuevo usuario llamará a la creación de este

Revalidate – Para un usuario autenticado, devolverá un nuevo “token”.

Los servicios de autenticación se encargan de, a través de “Spring Security”, validar los datos introducidos por el usuario para realizar las correspondientes acciones. Por una parte, a través del método “authenticate” se recibe un objeto “Authentication Request”, creado para el proyecto, que contiene un usuario y una contraseña. A través de “Spring Security” y un servicio de “JWT” que se comentará más adelante, se autentica el usuario dentro del contexto de la aplicación y se le genera un “token” para que durante tiempo limitado no necesite volver a autenticarse.

```
@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class AuthControllerImpl implements IAuthController{

    private final IAuthService authService;

    // /auth/login
    @PostMapping("/login")
    public ResponseEntity<AuthenticationResponse> login(@RequestBody AuthenticationRequest authenticationRequest) {
        return ResponseEntity.ok(authService.authenticate(authenticationRequest));
    }

    // auth/register
    @PostMapping("/register")
    public ResponseEntity<AuthenticationResponse> register(@RequestBody RegisterRequest registerRequest) {
        return ResponseEntity.ok(authService.register(registerRequest));
    }

    @GetMapping("/revalidate")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<AuthenticationResponse> revalidateToken() {
        return ResponseEntity.ok(authService.revalidateToken());
    }
}
```

Figura 43: Controlador de autenticación

El método de “register” se encarga de crear al usuario en la base de datos y además de realizar la autenticación directamente en el contexto además de generarle un “token” válido.

El método “revalidateToken” extrae los datos del “token” del usuario autenticado y devuelve otro completamente nuevo.

Para la generación de los “tokens” se ha creado una clase encargada de todas las acciones relacionadas con estos. Aquí se definen los tiempos de expiración de los “tokens” además de obtener la clave secreta que se utiliza para generarlos. Utilizando las clases que proporciona el paquete de “JWT” esta clase permite, sobre los “tokens”:

- Crear nuevos
- Comprobar si han expirado
- Validarlos
- Extraer la información relacionada

Por último se encuentra en el paquete una clase que implementa la interfaz “UserDetailsService”. Implementar esta interfaz permite definir un método a través del cual se obtiene un objeto “UserDetails”, necesario para operar la autenticación en el contexto, utilizando un nombre de usuario. En caso de no existir tal usuario lanzará un error.

Exercise

Este controlador maneja todas las peticiones relacionadas con los ejercicios. Aquí se encuentran métodos básicos de estos como pueden ser obtener, crear, actualizar y borrar ejercicios. Todas las acciones son permitidas por un usuario autenticado menos las de borrar. Con la notación “@PreAuthorize” se limita la entrada al método para las peticiones realizadas por usuarios sin el rol “Admin”.

En el servicio asociado encontramos unas funciones básicas interrelacionadas con el determinado método del repositorio. Los repositorios extienden de la clase “JpaRepository” la cual incluye una serie de métodos como pueden ser encontrar por “id” o guardar un objeto. Tiene sus correspondientes comprobaciones para que en caso de no encontrar el usuario/usuarios deseados lance un error.

```
@RestController
@RequestMapping("/exercises")
@RequiredArgsConstructor
public class ExerciseControllerImpl implements IExerciseController {

    private final IExerciseService exerciseService;

    @GetMapping
    public List<Exercise> findAll() {
        return exerciseService.findAll();
    }

    @GetMapping("/{id}")
    public Exercise findById(@PathVariable Long id) {
        return exerciseService.findById(id);
    }

    @GetMapping("/likeExample")
    public List<Exercise> findAllByExample(@RequestBody Exercise exercise) {
        return exerciseService.findAllByExample(exercise);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Exercise save(@RequestBody Exercise exercise) {
        return exerciseService.save(exercise);
    }

    @PostMapping("/saveList")
    @ResponseStatus(HttpStatus.CREATED)
    public List<Exercise> saveAll(@RequestBody List<Exercise> exercises) {
        return exerciseService.saveAll(exercises);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasAuthority('ADMIN')")
    public void deleteById(@PathVariable Long id) {
        exerciseService.deleteById(id);
    }

    @DeleteMapping("/likeExample")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasAuthority('ADMIN')")
    public void deleteByExample(@RequestBody Exercise exercise) {
        exerciseService.deleteByExample(exercise);
    }

    @PutMapping
    public Exercise update(@RequestBody Exercise exercise) {
        return exerciseService.update(exercise);
    }
}
```

```
@Service
@RequiredArgsConstructor
public class ExerciseServiceImpl implements IExerciseService {

    private final ExerciseRepository exerciseRepository;

    @Override
    public List<Exercise> findAll() {
        return exerciseRepository.findAll();
    }

    @Override
    public Exercise findById(Long id) {
        return exerciseRepository.findById(id).orElseThrow(() -> new RuntimeException(HttpStatus.NOT_FOUND));
    }

    @Override
    public List<Exercise> findAllByExample(Exercise exercise) {
        return exerciseRepository.findAll(exampleOf(exercise));
    }

    @Override
    public Exercise save(Exercise exercise) {
        return exerciseRepository.save(exercise);
    }

    @Override
    public List<Exercise> saveAll(List<Exercise> exercises) {
        return exerciseRepository.saveAll(exercises);
    }

    @Override
    public void deleteById(Long id) {
        exerciseRepository.deleteById(id);
    }

    @Override
    public void deleteByExample(Exercise exercise) {
        List<Exercise> exercisesToDelete = findAllByExample(exercise);
        if (exercisesToDelete.isEmpty())
            throw new RuntimeException(HttpStatus.NOT_FOUND);
        exerciseRepository.deleteAll(exercisesToDelete);
    }

    @Override
    public Exercise update(Exercise exercise) {
        findById(exercise.getId());
        return exerciseRepository.save(exercise);
    }
}
```

Figura 44: Servicio de ejercicio

Figura 45: Controlador de ejercicio

Record

El siguiente controlador se encarga de el manejo básico de todo lo que tenga que ver con los “Records” de un usuario de la misma forma que el anterior controlador. En este caso los usuarios solamente tienen acceso a añadir rutinas que estén asociados a ellos. Los “Records” consisten en un histórico de todas las rutinas del usuario con información sobre los ejercicios realizados y la rutina asociada. Estas funcionalidades no están implementadas en la aplicación móvil, pero se añadirán en futuros parches.

El servicio que corresponde con este controlador es muy parecido al servicio mencionado anteriormente. Las diferencias radican a la ahora de guardar un “record” para un usuario normal ya que, para usuarios normales, al tener que estar asociado a un usuario al ser creado, primeramente, comprueba que ese usuario existe y de ser así, añade la rutina a la lista de rutinas del usuario. El resto de métodos brindan las acciones básicas.

```
@Service
@RequiredArgsConstructor
public class RecordServiceImpl implements IRecordService {

    private final RecordRepository recordRepository;
    private final UserService userService;

    @Override
    public List<Record> findAll() {
        return recordRepository.findAll();
    }

    @Override
    public Record findById(Long id) {
        return recordRepository.findById(id).orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
    }

    @Override
    public List<Record> findByIdByExample(Record record) {
        return recordRepository.findAll(exampleOf(record));
    }

    @Override
    public Record save(Record record, Long id) {
        Record savedRecord = recordRepository.save(record);
        User user = userService.findById(id);
        List<Record> userRecordList = user.getRecords();
        userRecordList.add(savedRecord);
        user.setRecords(userRecordList);
        userService.update(user);
        return savedRecord;
    }

    @Override
    public List<Record> saveAll(List<Record> records) {
        return recordRepository.saveAll(records);
    }

    @Override
    public void deleteById(Long id) {
        recordRepository.delete(findById(id));
    }

    @Override
    public void deleteByExample(Record record) {
        List<Record> recordsToDelete = findAllByExample(record);
        if (recordsToDelete.isEmpty()) {
            throw new ResponseStatusException(HttpStatus.NOT_FOUND);
        }
        recordRepository.deleteAll(recordsToDelete);
    }

    @Override
    public Record update(Record record) {
        Record findById(record.getId());
        return recordRepository.save(record);
    }

    @Override
    public Record saveForAuthenticatedUser(Record record) {
        Record savedRecord = recordRepository.save(record);
        User user = userService.findAuthenticatedUser();
        List<Record> userRecordList = user.getRecords();
        userRecordList.add(savedRecord);
        user.setRecords(userRecordList);
        userService.update(user);
        return savedRecord;
    }
}
```

Figura 47: Servicio de "Record"

```
@RestController
@RequestMapping("/records")
@RequiredArgsConstructor
public class RecordControllerImpl implements IRecordController {

    private final IRecordService recordService;

    @GetMapping
    @PreAuthorize("hasAuthority('ADMIN')")
    public List<Record> findAll() {
        return recordService.findAll();
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAuthority('ADMIN')")
    public Record findById(@PathVariable Long id) {
        return recordService.findById(id);
    }

    @PostMapping("/likeExample")
    @PreAuthorize("hasAuthority('ADMIN')")
    public List<Record> findByIdByExample(@RequestBody Record record) {
        return recordService.findByIdByExample(record);
    }

    @PostMapping("/{id}")
    @ResponseStatus(HttpStatus.CREATED)
    @PreAuthorize("hasAuthority('ADMIN')")
    public Record save(@RequestBody Record record, @PathVariable Long id) {
        return recordService.save(record, id);
    }

    @PostMapping("/myUser")
    @ResponseStatus(HttpStatus.CREATED)
    public Record saveForAuthenticatedUser(@RequestBody Record record) {
        return recordService.saveForAuthenticatedUser(record);
    }

    @PostMapping("/saveList")
    @ResponseStatus(HttpStatus.CREATED)
    @PreAuthorize("hasAuthority('ADMIN')")
    public List<Record> saveAll(@RequestBody List<Record> records) {
        return recordService.saveAll(records);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasAuthority('ADMIN')")
    public void deleteById(@PathVariable Long id) {
        recordService.deleteById(id);
    }

    @DeleteMapping("/likeExample")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasAuthority('ADMIN')")
    public void deleteByExample(@RequestBody Record record) {
        recordService.deleteByExample(record);
    }

    @PutMapping
    @PreAuthorize("hasAuthority('ADMIN')")
    public Record update(@RequestBody Record record) {
        return recordService.update(record);
    }
}
```

Figura 46: Controlador de "Record"

Routine

Este controlador se encarga de gestionar las rutinas. No teniendo ninguna particularidad especial con respecto a los anteriores controladores, este puede realizar las acciones básicas de un “CRUD”(Create – Remove – Update – Delete).

El servicio es también simple como el controlador. Permite las opciones mencionadas anteriormente. Básicamente es el intermediario necesario entre el controlador y el servicio.

```
@RestController
@RequestMapping("/routines")
@RequiredArgsConstructor
public class RoutineControllerImpl implements IRoutineController {

    private final IRoutineService routineService;

    @GetMapping
    public List<Routine> findAll() {
        return routineService.findAll();
    }

    @GetMapping("/{id}")
    public Routine findById(@PathVariable Long id) {
        return routineService.findById(id);
    }

    @GetMapping("/likeExample")
    public List<Routine> findAllByExample(@RequestBody Routine routine) {
        return routineService.findAllByExample(routine);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Routine save(@RequestBody Routine routine) {
        return routineService.save(routine);
    }

    @PostMapping("/saveList")
    @ResponseStatus(HttpStatus.CREATED)
    public List<Routine> saveAll(@RequestBody List<Routine> routines) {
        return routineService.saveAll(routines);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasAuthority('ADMIN')")
    public void deleteById(@PathVariable Long id) {
        routineService.deleteById(id);
    }

    @DeleteMapping("/likeExample")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasAuthority('ADMIN')")
    public void deleteByExample(@RequestBody Routine routine) {
        routineService.deleteByExample(routine);
    }

    @PutMapping
    public Routine update(@RequestBody Routine routine) {
        return routineService.update(routine);
    }
}
```

```
@Service
@RequiredArgsConstructor
public class RoutineServiceImpl implements IRoutineService {

    private final RoutineRepository routineRepository;

    @Override
    public List<Routine> findAll() {
        return routineRepository.findAll();
    }

    @Override
    public Routine findById(Long id) {
        return routineRepository.findById(id).orElseThrow(() -> new RuntimeException(HttpStatus.NOT_FOUND));
    }

    @Override
    public List<Routine> findAllByExample(Routine routine) {
        return routineRepository.findAll(Example.of(routine));
    }

    @Override
    public Routine save(Routine routine) {
        return routineRepository.save(routine);
    }

    @Override
    public List<Routine> saveAll(List<Routine> routines) {
        return routineRepository.saveAll(routines);
    }

    @Override
    public void deleteById(Long id) {
        routineRepository.deleteById(id);
    }

    @Override
    public void deleteByExample(Routine routine) {
        List<Routine> routinesToDelete = findAllByExample(routine);
        if (routinesToDelete.isEmpty())
            throw new RuntimeException(HttpStatus.NOT_FOUND);

        routineRepository.deleteAll(routinesToDelete);
    }

    @Override
    public Routine update(Routine routine) {
        findById(routine.getId());
        return routineRepository.save(routine);
    }
}
```

Figura 48: Servicio de rutina

Figura 49: Controlador de rutina

User

Este es el controlador más protegido de todos. Al ser información muy sensible, cualquier usuario que no sea administrador solamente podrá acceder a dos “endpoints”. Estos “endpoints” le permiten al usuario obtener la información correspondiente al “token” validado del usuario y a actualizar también la información de este.

Entre los métodos del servicio de usuario, hay dos que se diferencian de los básicos. Primero el “findAuthenticatedUser”, el cual permite obtener los datos del usuario encontrado en el contexto de “Spring”. Por otra parte el método para actualizar a este usuario, el cual solamente permite modificar una serie de parámetros, asegurando así que no se modifican campos que deberían ser inmutables.

Este es el único caso donde un repositorio tiene una función agregada a mayores de las que “JpaRepository” brinda. El método definido aquí es gestionado por el “framework”, obteniendo un objeto “Optional”, el cual envuelve a un objeto “User”, a través del nombre de usuario.

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
}
```

Figura 51: Repositorio de usuario

```
@RestController
@RequestMapping("/users")
@RequiredArgsConstructor
public class UserControllerImpl implements IUserController {

    private final IUserService userService;

    @GetMapping
    @PreAuthorize("hasAuthority('ADMIN')")
    public List<User> findAll() {
        return userService.findAll();
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAuthority('ADMIN')")
    public User findById(@PathVariable Long id) {
        return userService.findById(id);
    }

    @GetMapping("/myUser")
    public User findAuthenticatedUser() {return userService.findAuthenticatedUser();}

    @PostMapping("/likeExample")
    @PreAuthorize("hasAuthority('ADMIN')")
    public List<User> findAllByExample(@RequestBody User user) {
        return userService.findAllByExample(user);
    }

    @PostMapping("/register")
    @ResponseStatus(HttpStatus.CREATED)
    @PreAuthorize("hasAuthority('ADMIN')")
    public User save(User user) {
        return userService.save(user);
    }

    @PostMapping("/saveList")
    @ResponseStatus(HttpStatus.CREATED)
    @PreAuthorize("hasAuthority('ADMIN')")
    public List<User> saveAll(@RequestBody List<User> users) {
        return userService.saveAll(users);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasAuthority('ADMIN')")
    public void deleteById(@PathVariable Long id) {
        userService.deleteById(id);
    }

    @DeleteMapping("/likeExample")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @PreAuthorize("hasAuthority('ADMIN')")
    public void deleteByExample(@RequestBody User user) {
        userService.deleteByExample(user);
    }

    @PutMapping
    @PreAuthorize("hasAuthority('ADMIN')")
    public User update(@RequestBody User user) {
        return userService.update(user);
    }

    @PatchMapping("/myUser")
    public User updateAuthenticatedUser(@RequestBody User user) {
        return userService.updateAuthenticatedUser(user);
    }
}
```

Figura 50: Controlador de usuario

Configuraciones

Esta clase es necesaria para el uso personalizado de “Spring Security”. Esta clase se introduce en el contexto de la aplicación para sobrescribir las configuraciones por defecto del filtrado de las peticiones. Además se ha utilizado esta clase para definir una serie de métodos que devuelven objetos que se introducen también en el contexto de la aplicación y son necesarias para cumplir aspectos de la seguridad.

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityJavaConfig{

    private final JwtAuthFilter jwtAuthFilter;
    private final MyUserDetailsService myUserDetailsService;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            .cors().disable()
            .csrf().disable()

            .authorizeHttpRequests(requests -> requests
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers("/records/**").hasAuthority("ADMIN")
                .anyRequest().authenticated()
            )
            .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            .and()
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)

        ;

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
        authProvider.setUserDetailsService(myUserDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}
```

Figura 52: Clase de configuración de "Spring Security"

La primera es “SecurityFilterChain”. Aquí se definen todas las políticas de filtrado. En este caso se permiten todas las peticiones dirigidas a “/auth”, las cuales tienen como objetivo autenticar al usuario. Los “endpoints” relacionados con los “records” se han limitado únicamente a usuarios con el rol de administrador ya que no están implementados.

Después, se bloquea el acceso al resto de “endpoints” a todo usuario que no esté autenticado. También se indica la política “Stateless”, la cual establece que no se debe conservar ningún estado de conexión. Por último, se define cual es el proveedor de autenticación que se explicará ahora y se añade un filtro personalizado que también se comentará.

El “Bean” de “PasswordEncoder” define la clase a utilizar para codificar donde se llame un “PasswordEncoder”.

“AuthenticationProvider” define que el proveedor será un objeto “DaoAuthenticationProvider”, se configura estableciendo el “UserDetailsService” que definimos previamente y por último se establece el “PasswordEncoder” que acabamos de mencionar.

Por último, se define el “Bean” de “AuthenticationManager”, el cual se deja por defecto. Se establece por si es necesario realizar configuraciones a mayores.

En la clase “JwtAuthFilter” se extiende la clase “OncePerRequestFilter”. Esta clase obliga a sobrescribir un método que se va a ejecutar para filtrar las peticiones. En este caso se obtiene el “token” de la petición. En caso de existir y ser válido se autenticará al usuario dentro del contexto para posteriormente realizar las operaciones que solicitan la petición.

```
@Component
@RequiredArgsConstructor
@Log4j2
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final MyUserDetailsService myUserDetailsService;

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {

        final String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION);
        final String username;
        String jwtToken;

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        jwtToken = authHeader.substring(7);

        try {
            username = jwtService.extractUsername(jwtToken);
        } catch (ExpiredJwtException e) {
            log.warn("Expired JWT token");
            filterChain.doFilter(request, response);
            return;
        }

        if (username == null || SecurityContextHolder.getContext().getAuthentication() != null) {
            filterChain.doFilter(request, response);
            return;
        }

        UserDetails userDetails = myUserDetailsService.loadUserByUsername(username);

        for (GrantedAuthority a : userDetails.getAuthorities()) {
            System.out.println(a);
        }

        if (!jwtService.validateToken(jwtToken, userDetails)) {
            filterChain.doFilter(request, response);
            return;
        }

        UsernamePasswordAuthenticationToken authToken =
            new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());

        authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(authToken);

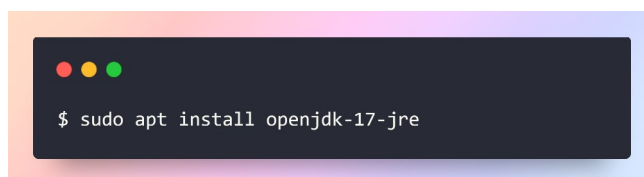
        filterChain.doFilter(request, response);
    }
}
```

Figura 53: Clase de filtrado de peticiones “Http”

Manual Administrador

Para utilizar la aplicación móvil no se necesita instalar el “API”, igualmente se va a proceder a explicar el proceso a seguir para instalarla en caso de que se quiera utilizar la aplicación y el “API” oficial no esté en funcionamiento. Esta instalación se realizará en una máquina “Ubuntu” sin interfaz gráfica.

Para comentar necesitaremos una máquina que tenga instalado el Java. En este caso se utilizará Java 17. En máquinas “Windows” se puede descargar en la página oficial. Para instalarlo se deberá de abrir una terminal e introducir el siguiente comando:

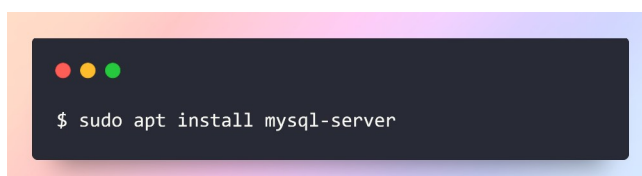


```
$ sudo apt install openjdk-17-jre
```

Figura 54: Comando de instalación de Java

Una vez realizada la operación Java estará instalado con éxito en nuestro equipo.

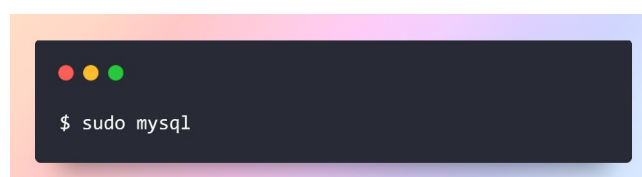
El siguiente punto es la instalación de “Mysql”. Para máquinas “Windows”, “Mysql” viene incluido en instalaciones como “Wamp”, “Xamp” o “Mysql Workbench”. Es importante tener en cuenta que siguiendo este tutorial, la instalación de este programa se realizará de forma no segura, es decir, no se debe usar para guardar datos sensibles pues la instalación está diseñada para una demo. Dicho esto, en la terminal previamente abierta se ejecutará el siguiente comando:



```
$ sudo apt install mysql-server
```

Figura 55: Comando de instalación de “Mysql”

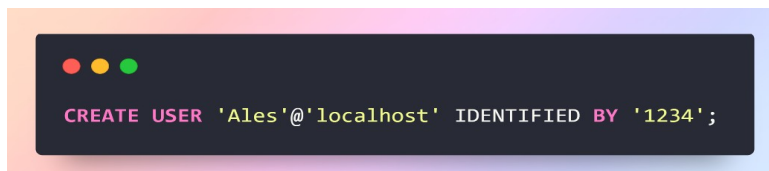
Una vez hecho esto, “Mysql” estará instalado en nuestra máquina. Para que la aplicación pueda conectarse exitosamente se deberá crear un nuevo usuario y la base de datos. Para esto hay que acceder a la aplicación. Para acceder se ejecutará:



```
$ sudo mysql
```

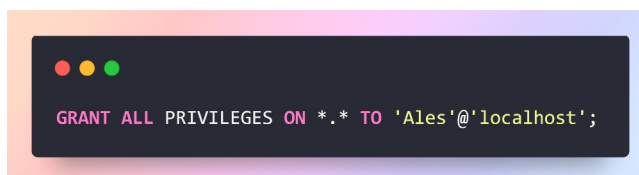
Figura 56: Comando para conexión con “Mysql”

Para acceder en “Windows” se utilizará el “software” instalado anteriormente y utilizar la herramienta que permita ejecutar peticiones. Una vez dentro de “Mysql”, deberemos escribir y los siguientes comandos, a fin de crear el usuario deseado y concederle los permisos necesarios:



```
CREATE USER 'Ales'@'localhost' IDENTIFIED BY '1234';
```

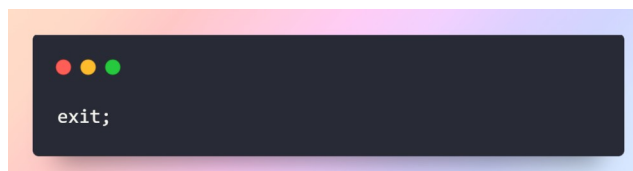
Figura 57: Comando para creación de usuario en "Mysql"



```
GRANT ALL PRIVILEGES ON *.* TO 'Ales'@'localhost';
```

Figura 58: Comando para conceder permisos en "Mysql"

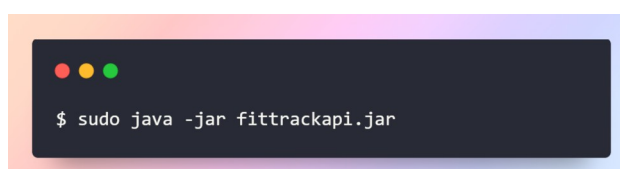
Para crear la base de datos, el proyecto trae consigo un archivo con formato “SQL”. Se deberá copiar el contenido de ese archivo abriéndolo con el bloc de notas y se deberá pegar en “Mysql”. Tras pegarlo se presionará “Enter” a fin de ejecutar la última línea y se saldrá de la aplicación con el comando:



```
exit;
```

Figura 59: Comando para salir de "Mysql"

Para terminar y poner a funcionar la aplicación. En el repositorio de “Github” se encuentra un archivo con extensión “jar”, este archivo es la propia aplicación de “FitTrackApi”. Siguiendo en la terminal una última vez, ejecutaremos en la ubicación donde hayamos puesto el archivo el siguiente comando:



```
$ sudo java -jar fittrackapi.jar
```

Figura 60: Comando para iniciar "API"

En “Windows” se ejecutará sin la palabra “sudo”. Tras haber hecho esto aparecerán una serie “Logs” que darán información de la aplicación e indicarán que esta ya está en marcha.

Manual Usuario

Aquí se explicará la instalación y el uso de la aplicación. Para la instalación de “FitTrack Mobile” es necesario un dispositivo “Android”. En el repositorio se encontrará un archivo “.apk”, este archivo es la aplicación. Para este tutorial se usará un “Google Pixel 5” con “Android” 13.

Para instalarlo se deberá ubicar el archivo dentro de nuestro dispositivo. Habitualmente se encontrará en descargas si ha sido descargado desde el móvil. Si ha sido copiado de alguna forma externa habrá que navegar hasta la carpeta donde se copió.

Al pulsar sobre el “APK” debería iniciar el proceso de instalación. Dependiendo del dispositivo hay una serie de protecciones que evitan instalar “software” potencialmente malicioso. Al ser una aplicación desconocida el sistema dará una señal de alerta. En mi caso es únicamente texto emergente.

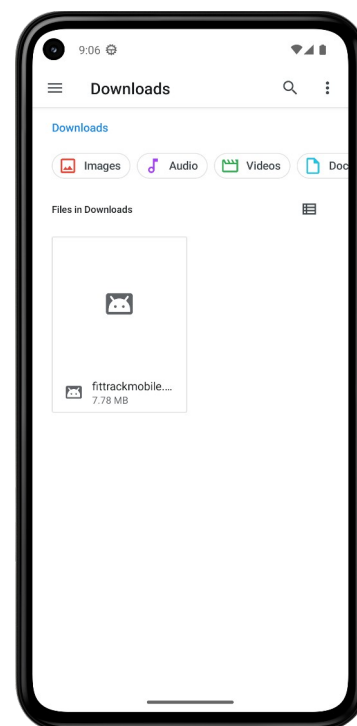


Figura 61: Pantalla de explorador de archivos del móvil

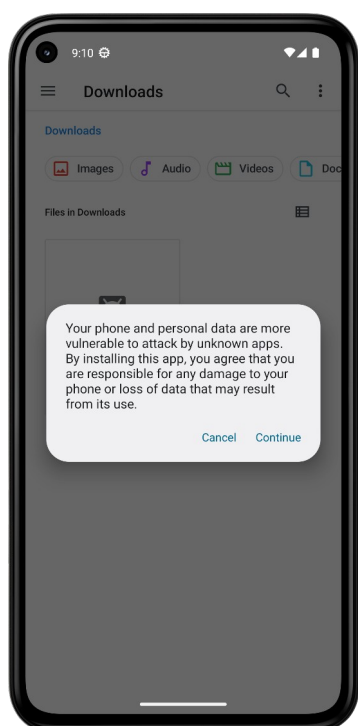


Figura 62: Mensaje de confirmación de instalación de aplicación desconocida

Si en el dispositivo en el que se intenta instalar no es así, probablemente de instrucciones claras de como permitir este tipo de instalación.

Una vez aceptado todo se instalará la aplicación y ya estará lista para su uso.

Una vez dentro de la aplicación se abrirá una ventana de “Login”. En esta ventana se realizará el inicio de sesión o en su defecto se podrá abrir la pagina de registro y realizar un registro de un nuevo usuario. Para cambiar entre las dos pantallas se puede hacer clic en el texto resaltado en morado que se encuentra abajo a la derecha.

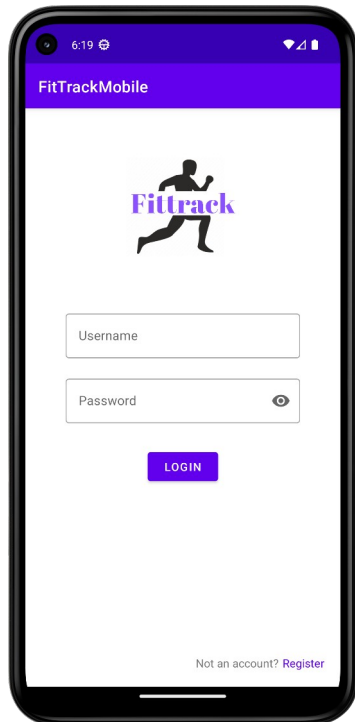


Figura 63: Pantalla de "Login"

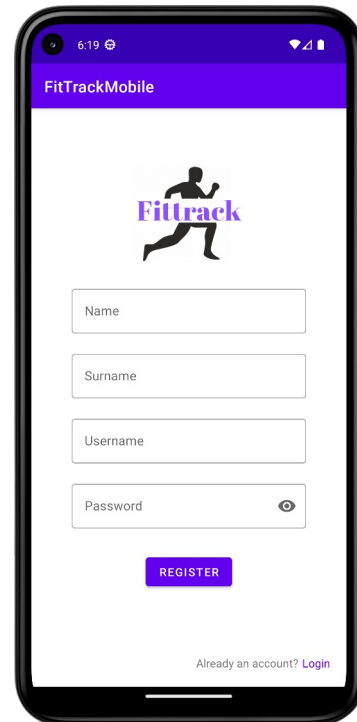


Figura 64: Pantalla de registro

Una vez rellenados todos los datos se pulsará el botón correspondiente y, si no hay ningún problema, se pasará a la pantalla principal de la aplicación

Esta pantalla se abrirá mostrándo una lista de las rutinas que el usuario tiene actualmente. En caso de no haber rutinas solamente se verá el botón de añadir rutinas. A mayores se podrá ver en la parte inferior una barra de navegación. Esta barra se usará para navegar entre las tres pantallas principales de la aplicación.

Para usar la primera pantalla, “home”, solamente habrá que crear una nueva rutina con el botón mencionado anteriormente o pulsar en una rutina ya existente.

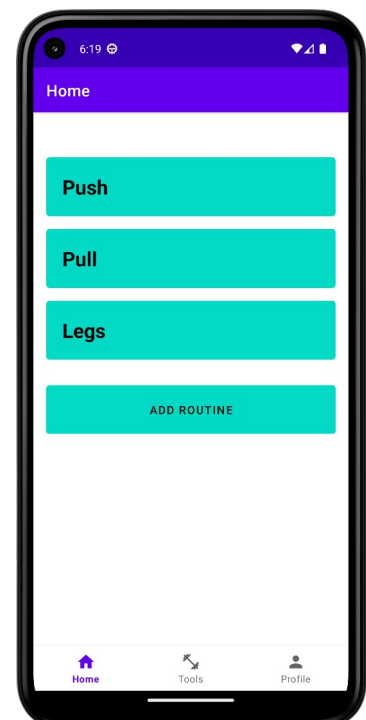


Figura 65: Pantalla de "Home"

Tras pulsar una rutina ya existente se abrirá la vista de rutina. En caso de haber creado una nueva se abrirá la misma vista pero en modo edición y sin ejercicios. Aquí se podrá observar una lista de los ejercicios asociados a la rutina y un botón para habilitar el modo edición. Tras pulsar el botón se mostrarán los botones de gestión de la rutina y unos iconos en cada ejercicio para eliminarlos de la rutina.

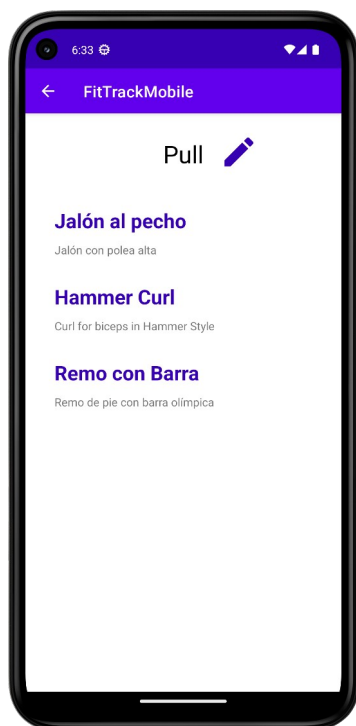


Figura 66: Pantalla de rutina

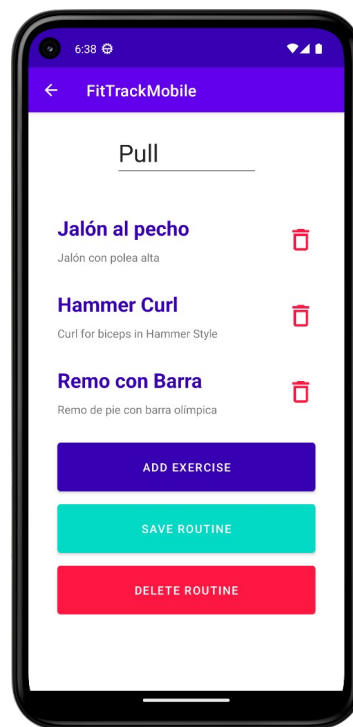


Figura 67: Pantalla de rutina en modo edición

Para guardar los cambios realizados se pulsará el botón “*Save Routine*”, para eliminar la rutina el botón “*Delete Routine*” y para añadir más ejercicios a la lista “*Add Exercise*”. Para salir del modo edición se deberá pulsar la flecha ubicada en la parte superior izquierda y pulsarla nuevamente si se desea salir de la vista de rutina.

Si pulsamos el botón para añadir un ejercicio se mostrará una lista con todos los ejercicios. Para añadirlo habrá que pulsar sobre el que se desea añadir y regresaremos a la edición con ese ejercicio en la lista.

Para crear un ejercicio nuevo solamente se deberá pulsar el botón que aparece debajo de la lista de ejercicios y se abrirá una nueva pantalla para crearlo. Para salir sin añadir un ejercicio se pulsará en la flecha superior izquierda.

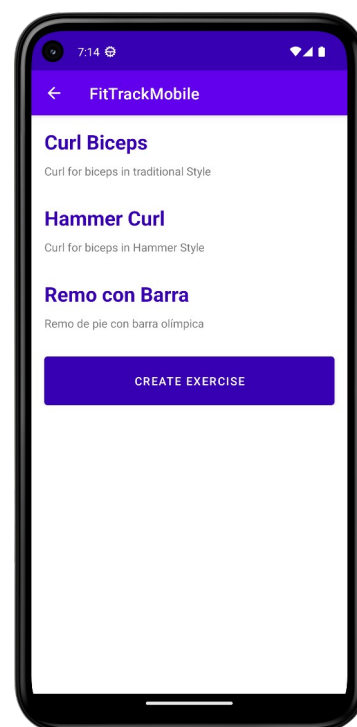


Figura 68: Pantalla para añadir ejercicio

En la pantalla de creación de un ejercicio se encontrarán diferentes campos para crearlo. Aquí se podrá configurar el nombre del ejercicio, una descripción, el tipo de ejercicio y los músculos que intervienen en él.

Para añadir músculos se deberán seleccionar en el desplegable y pulsar el botón “Add”, una vez hecho esto, el músculo se añadirá a una lista que se encontrará justo debajo.

Una vez configurado al gusto, se podrá guardar pulsando el botón de la parte inferior, que regresará a la lista de ejercicios con el nuevo ejercicio añadido. Para salir sin guardarlo se pulsará la flecha de arriba a la izquierda.

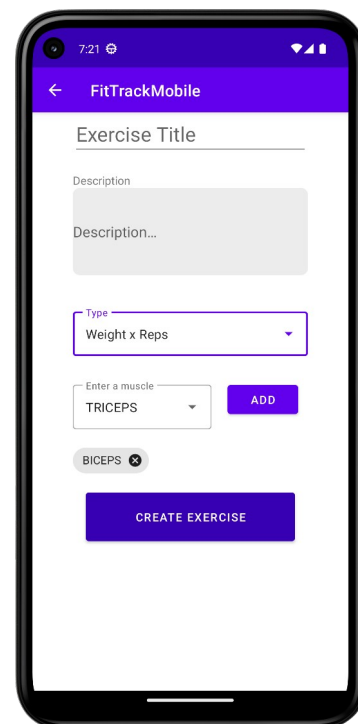


Figura 69: Pantalla de creación de ejercicio

Volviendo a la pantalla principal y como tercera opción del menú de navegación se encuentra el perfil de usuario. Este muestra toda la información relevante para el usuario y permite editarla o cerrar sesión de la aplicación. Para editarlo se pulsará el botón “Edit Profile”. Una vez pulsado se abrirá la pantalla de edición de perfil.

En esta pantalla se podrán modificar los campos deseados y guardarlos pulsando el botón inferior. Para salir sin guardar se deberá pulsar la flecha superior izquierda. Los cambios se sincronizarán al instante, en caso de que haya algún error, no se guardarán.

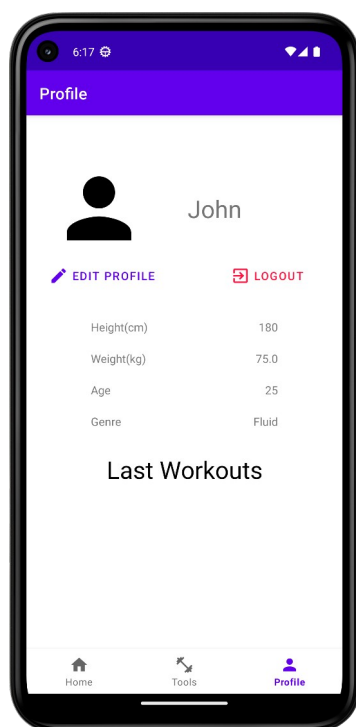


Figura 70: Pantalla de perfil de usuario

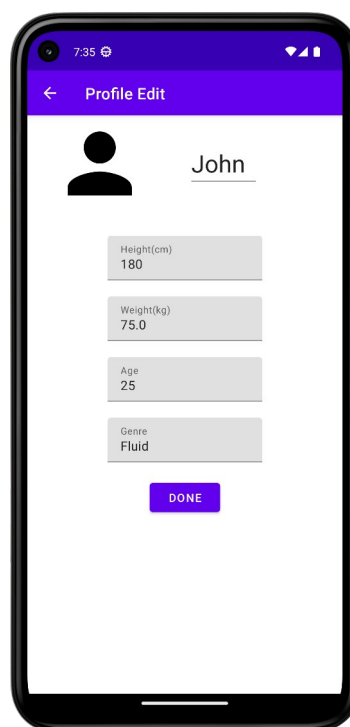


Figura 71: Pantalla de edición del perfil de usuario

FitTrack – Aplicación de seguimiento deportivo.

La última pantalla que se encontrará a través del menú de navegación es la pantalla de herramientas. En esta pantalla se encuentran cuatro herramientas. Estas herramientas están diseñadas para complementar el uso habitual de la aplicación. Para usar cada herramienta solamente se deberá pulsar el icono correspondiente.

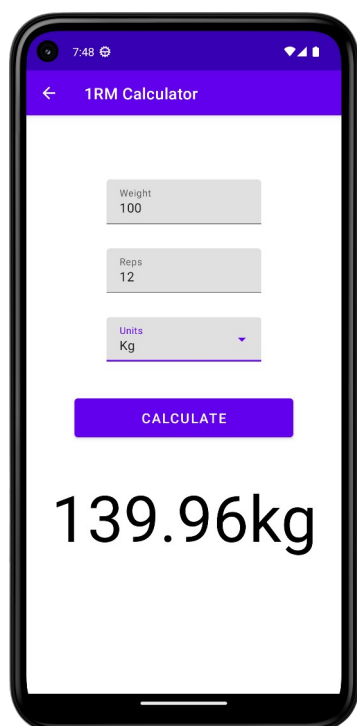


Figura 73: Pantalla de "1RM Calculator"

La herramienta "1RM Calculator" permitirá calcular el mayor peso a levantar en un ejercicio tras introducir el peso levantado por la cantidad de repeticiones.

La calculadora de calorías obtendrá el número de calorías que una persona debe consumir por el gasto diario del propio cuerpo y por el gasto añadido del ejercicio y demás acciones. Para esto se necesitará el peso, altura, género y cantidad de actividad deportiva. Algunos datos pueden estar previamente introducidos con la información que el usuario tenga en su perfil.

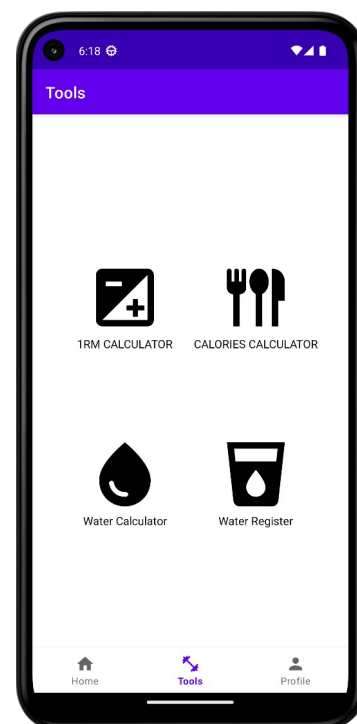


Figura 72: Pantalla de herramientas

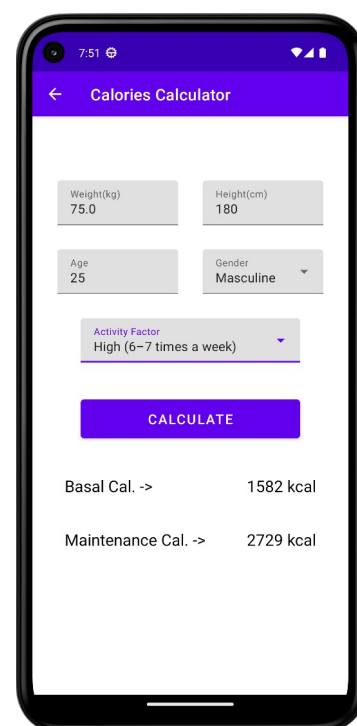


Figura 74: Pantalla de calculadora de calorías

FitTrack – Aplicación de seguimiento deportivo.

La calculadora de agua calculará el agua necesaria para una persona de forma diaria utilizando el peso y la cantidad de deporte.



Figura 76: Pantalla de registro de agua

El registro de agua permitirá llevar una cuenta de la cantidad de vasos de agua bebidos a lo largo del día. Solamente se pueden sumar y restar pulsando los botones y la cantidad se reinicia cada día.

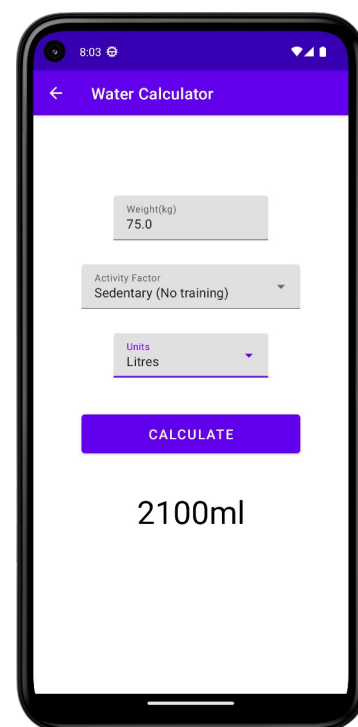


Figura 75: Pantalla de calculadora de agua

Para salir de estas pantallas se deberá pulsar el botón de flecha ubicado arriba a la izquierda.

Para desinstalar la aplicación se deberá mantener pulsado sobre el icono de esta, pulsar el botón de “App info” y en la siguiente pantalla seleccionar desinstalar.

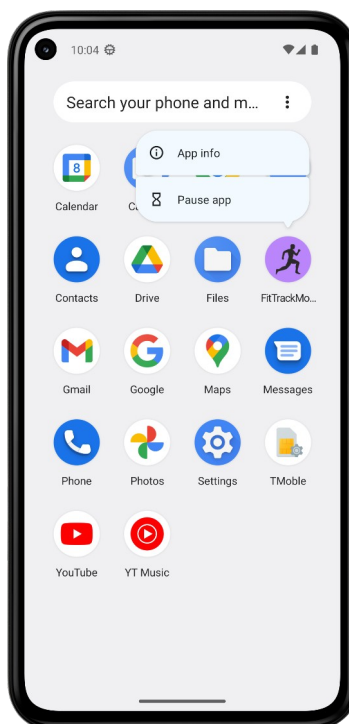


Figura 77: Pantalla de aplicaciones del dispositivo

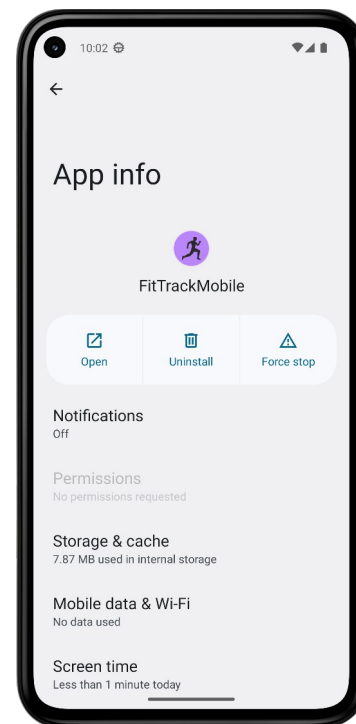


Figura 78: Menu de información de la aplicación

Viabilidad tecno-económica.

Para el análisis de la viabilidad tecno-económica se tendrán en cuenta los siguientes puntos:

- Coste de personal
- Coste de licencias
- Coste de creación y mantenimiento de servicios
- Costes asociados a la creación de una empresa
- “Marketing”

Para comenzar a desarrollar el proyecto se necesitará a un equipo mediano de desarrolladores. Pensando en los perfiles que se requerirán se puede estimar un mínimo inicial de tres personas. Una de esas personas inicialmente, ya que es el dueño de la compañía, cobraría lo mínimo indispensable. Teniendo en cuenta los gastos promedio en desarrolladores “Senior” en España, al mes habría un gasto relacionado a esas personas de 3.038€ / persona incluyendo todos los gastos asociados. Además se deberán entregar equipos a los trabajadores con un gasto único inicial de 3.000€.

En licencias, para comenzar un proyecto no será necesario realizar un gasto ya que lo que necesitará el equipo para desarrollar podrá ser seleccionado entre las alternativas gratuitas del mercado aunque carezcan de la misma potencia.

Para montar una aplicación es necesario un servidor el cual se contratará en la nube. No habrán oficinas físicas por lo que lo único que habrá que mantener y pagar serán estos servicios. Para comenzar el proyecto y para seguir con la línea de la demo, se contratarían estos servicios en “Oracle Cloud”. El coste de una instancia virtual básica tiene un coste de 54\$ mensuales, alrededor de 50€. Aquí se desplegaría tanto el “Api” como la base de datos inicialmente.

El planteamiento inicial de “Marketing” será la creación de videos virales y tendencias que puedan haber a fin de llegar al mayor público posible con el menor coste. Según la empresa creciese se valorarían publicidades pagadas a “influencers” o en páginas deportivas.

Se necesitará darse de alta como autónomo. Al ser la primera vez el coste inicial es de 80€ durante 24 meses.

FitTrack – Aplicación de seguimiento deportivo.

Con todo esto habrán unos costes iniciales estimados de 9.206€ y unos gastos mensuales de 6126€

Para la facturación la aplicación tendrá un sistema de cobro por actividades profesionales. Debido al enfoque de desarrollo para grupos y profesionales del entorno deportivo, se podrá ofrecer un servicio personalizado y adaptado para este tipo de usuarios. Este servicio tendrá un coste añadido que dependerá del trabajo.

El coste se dividirá en tramos, comenzando el primer tramo en una suscripción de 27,99€ mensuales para pequeños profesionales como entrenadores de equipos juveniles o entrenadores personales con pocos clientes. Los tramos subirán en función de cantidad de deportistas por entrenador o de magnitud deportiva, cobrando más a entrenadores olímpicos por ejemplo. Este precio puede llegar a los 239,99€. Para este tipo de usuarios se darán servicios añadidos e integraciones con distintos sistemas relacionados con ese rango de nivel deportivo como pueden ser seguros sanitarios y demás.

Para que la aplicación sea rentable se necesitarían un mínimo de 329 usuarios profesionales básicos o de 39 usuarios profesionales “*Premium*”. A partir de ese número de usuarios los beneficios serán variables en función de la fama de la aplicación.

Estos costes dependerán del ciclo de vida de la aplicación y la cantidad de usuarios futura. El alcance es global ya que la aplicación está desarrollada en inglés. Se pueden además generar beneficios a través de futuros patrocinios o eventos.

Trabajo futuro.

Los siguientes puntos darán unas pinceladas de futuras implementaciones y desarrollos para conseguir los objetivos propuestos y mejorar el proyecto:

- Creación de una aplicación de escritorio para que los usuarios profesionales tengan acceso a las rutinas y resultados de todos los usuarios a su cargo. Este punto de los más importantes ya que al ser los usuarios de pago se necesitarán estas funcionalidades para expandir la aplicación
- Mejorar el sistema de seguimiento de rutinas añadiendo la posibilidad de generar un registro de la realización de una rutina, incluyendo duración, series por ejercicio y kilos y series totales.
- Añadir nuevas herramientas que puedan ser útiles para los usuarios y que incluyan la implementación de “*hardware*” relacionado con el deporte.
- Diseño del apartado social en el que los usuarios puedan compartir sus rutinas y resultados y puedan estar todos conectados.
- Recomendaciones basadas en trabajo pasado del usuario para generar recomendaciones de cambios en rutinas o en la realización de estas.
- Posibilidad de tener una rutina como activa para llevar el seguimiento en tiempo real de la rutina que se este realizando en el momento.
- Esquemas y gráficos para representar el progreso del usuario
- Imágenes y explicaciones de la realización de cada ejercicio así como un mapa muscular, permitiendo al usuario comprobar con más exactitud los músculos a trabajar.
- Añadir notificaciones personalizadas para recordar al usuario diversas cosas del entreno o necesidades diarias como el mínimo de agua o calorías.
- Añadir un seguimiento de las comidas teniendo un histórico de las calorías diarias.
- Mucho más...

Conclusiones.

Este proyecto ha sido para mí especialmente enriquecedor. Cada paso que he dado me ha permitido ver el mundo del desarrollo de una forma mucho más cercana y con un punto de vista que nuevo para mí.

Tener de principio a fin esta labor me ha ayudado a ver hasta donde pueden llegar mis capacidades y a perder el miedo a la creación de cualquier proyecto que me proponga.

Si bien los objetivos iniciales eran mucho más optimistas de lo que el tiempo me ha permitido crear, los resultados obtenidos me han dejado gratamente satisfecho. Vivir esta experiencia al lado de gente en mi misma situación ha sido cuanto menos curioso y el intercambio de ideas le ha dado mucho a mi desarrollo.

La resolución de errores y la lluvia de ideas ha sido esencial ya que este proyecto me ha ido acompañando en mi día a día, siempre viendo nuevas cosas para añadir o formas de solucionar un problema concreto.

Este proyecto me ha servido para poner el comienzo de una larga carrera y de muchos otros proyectos que están por llegar, cada uno mejor que el anterior. He disfrutado mucho con este desarrollo y siempre lo recordaré con ilusión.

Biblioteca de recursos web y referencias.

Sgoliver.net, (2023): *Curso de programación android*. Disponible en: [enlace](#)

Agapito, A. (2020): *Spring Framework 5 + REST de cero a experto*. Disponible en: [enlace](#)